Chương 4

Các mô hình chuỗi hồi quy và tích chập

Cái toàn thể lớn hơn tổng các bộ phận của nó.

– Aristotle, 384–322 TCN [Ross, 1924]

Có thể Aristotle đã nghĩ, cũng có thể đã không nghĩ, tới các hiện tượng ngôn ngữ khi phát biểu ý tưởng này, nhưng đó đúng là điều chúng tôi muốn truyền tải trong chương này: một câu hay một cụm từ có ý nghĩa vượt xa một tập hợp từ đơn thuần. Mỗi từ tự nó đã mang nghĩa, nhưng khi các từ kết hợp với nhau thành câu hay cụm từ, ý nghĩa của tổng thể có thể phức tạp và đa dạng hơn rất nhiều. Điều này dẫn tới khía cạnh đẹp đẽ nhất của ngôn ngữ: con người có thể diễn đạt vô số ý nghĩa khác nhau chỉ bằng một tập hữu hạn các thành phần (ví dụ: từ hoặc ký tự).

Bản chất vô hạn và phụ thuộc mạnh vào ngữ cảnh của ngôn ngữ khiến việc mô hình hóa một chuỗi từ khó hơn nhiều so với việc mô hình hóa từng từ riêng lẻ. Một thách thức là nghĩa của một từ có thể thay đổi hoàn toàn tùy theo ngữ cảnh (*context*) của nó. Dựa trên khái niệm nhúng từ (*word embedding*), trong đó mỗi từ được biểu diễn bằng một vectơ số thực có số chiều thấp, "ý nghĩa" của một đơn vị ngôn ngữ có thể được mô hình hóa trong một không gian liên tục. Do đó, ta có thể mở rộng các phương pháp biểu diễn phân tán từ từng từ đơn lẻ lên toàn bộ chuỗi. Điều này thúc đẩy việc tìm hiểu các mô hình xử lý chuỗi từ có độ dài thay đổi theo một cách liên tục về bản chất.

Ở đây, chúng ta tìm hiểu cách tiếp cận tổng quát để học các biểu

diễn phân tán của chuỗi từ. Cụ thể, chúng ta xem xét mạng nơ-ron hồi quy (*recurrent neural network*) và mạng nơ-ron tích chập (*convolutional neural network*), vốn đã được dùng rộng rãi trong nhiều lĩnh vực, từ xử lý tiếng nói tới thị giác máy tính. Với đầu vào là ngôn ngữ tự nhiên, áp dụng các mô hình này cho ta một biểu diễn (*representation*) ở mức chuỗi. Biểu diễn này có thể là một vectơ số thực duy nhất, hoặc là một chuỗi các vectơ như vậy, trong đó mỗi vectơ ứng với một biểu diễn theo ngữ cảnh của một từ cụ thể trong chuỗi đầu vào. Các mô hình biểu diễn này, thường được gọi là bộ mã hóa (*encoder*), được dùng phổ biến trong nhiều hệ thống xử lý dữ liệu tuần tự. Chúng ta sẽ thấy một số ví dụ như vậy trong chương này.

Chú giải của người dịch: "Hồi quy" và "tích chập" ở đây nghĩa là gì?

Chữ *hồi quy* trong "mạng nơ-ron hồi quy" (*recurrent*) mang nghĩa "lặp đi lặp lại, quay vòng": mạng đọc từng từ một và mỗi bước lại dùng lại kết quả của bước trước — giống như khi đọc một câu, ta nhớ những gì đã đọc rồi cộng thêm từ mới. Nó *không* liên quan tới "hồi quy tuyến tính" (*regression*) trong thống kê. Còn *tích chập* (*convolution*) là phép "trượt" một cửa sổ nhỏ dọc theo chuỗi và tính toán trên từng nhóm vài từ liền nhau (sẽ trình bày ở Mục 4.4).

4.1 Phát biểu bài toán

Với nhiều ứng dụng NLP, mục tiêu của chúng ta là đưa ra dự đoán dựa trên một chuỗi đầu vào. Hãy xét lại bài toán phân loại (*classification*) văn bản đã nhắc tới ở Chương 1. Cho một văn bản có thể nói hoặc không nói về đồ ăn, ta muốn gán cho nó một trong hai lớp (ví dụ: Food (đồ ăn) hoặc Not-food (không phải đồ ăn)). Để làm điều này, một phương pháp phân loại phổ biến là biểu diễn văn bản dưới dạng một "túi" các đặc trưng (*feature*), ký hiệu là $\mathbf{H}$. Sau đó, mỗi lớp được gán một xác suất bằng một mô hình xác suất $\Pr(y|\mathbf{H})$. Lớp được dự đoán là lớp làm cực đại xác suất này: $\hat{y} = \arg\max_y \Pr(y|\mathbf{H})$.

Tuy đây là một quy trình chuẩn cho bài toán phân loại, ý tưởng đằng sau nó mô tả một bài toán rộng hơn nhiều. Một cách hình thức, gọi $\mathbf{w} =$

$w_1 \cdots w_m$ là một chuỗi từ[1]. Một hệ thống NLP ở mức chuỗi có thể được phát biểu như một hàm ánh xạ chuỗi đầu vào $\mathbf{w}$ thành một đầu ra $\mathbf{y}$ nào đó. Quá trình này có thể chia thành hai bước riêng biệt: biểu diễn (hay mã hóa) và dự đoán.

- **Biểu diễn** (hay **Mã hóa**). Bước này biến đổi chuỗi đầu vào $\mathbf{w}$ thành một biểu diễn đặc trưng $\mathbf{H}$ bằng một bộ mã hóa Encode$(\cdot)$:

$$\mathbf{H} = \text{Encode}(\mathbf{w}). \tag{4.1}$$

- **Dự đoán**. Một bộ dự đoán Predict$(\cdot)$ nhận $\mathbf{H}$ và sinh ra đầu ra:

$$\mathbf{y} = \text{Predict}(\mathbf{H}). \tag{4.2}$$

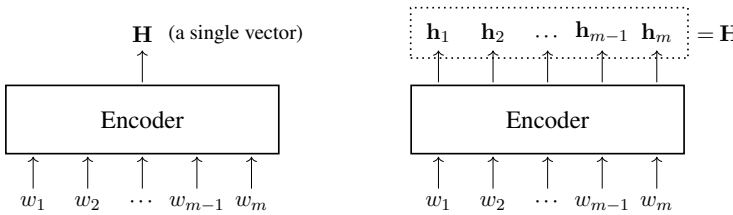
Một dạng đơn giản của $\mathbf{H}$ là một vectơ đặc trưng (*feature vector*). Chẳng hạn, $\mathbf{H}$ có thể là một tập các đặc trưng chỉ báo do con người thiết kế, được trích từ $\mathbf{w}$ (tạo thành một biểu diễn thưa nhiều chiều), hoặc một tập các số thực biểu thị các đặc trưng ẩn (tạo thành một biểu diễn dày đặc ít chiều). Trong NLP, một dạng phổ biến khác của $\mathbf{H}$ là một chuỗi các vectơ, trong đó mỗi vectơ $\mathbf{h}_i$ tương ứng với một từ đầu vào w_i (xem Hình 4.1). Trong trường hợp này, $\mathbf{h}_i$ có thể được xem là một biểu diễn theo ngữ cảnh của cả w_i lẫn ngữ cảnh xung quanh nó trong $\mathbf{w}$[2]. Sự tương ứng giữa $\mathbf{h}_i$ và w_i cho phép biểu diễn phân biệt được các vị trí khác nhau trong chuỗi và, quan trọng hơn, tự điều chỉnh năng lực mô hình hóa của nó cho các đầu vào có độ dài thay đổi.

Dạng của $\mathbf{y}$ phụ thuộc vào bài toán ta muốn giải. Ví dụ, trong bài toán phân loại, $\mathbf{y}$ là chỉ số của một lớp (hoặc một phân phối trên các lớp); trong bài toán hồi quy (*regression*), $\mathbf{y}$ là một số thực; còn trong dịch máy (*machine translation*), $\mathbf{y}$ là một chuỗi từ trong ngôn ngữ đích. Lưu ý rằng trong mô hình này, biểu diễn và dự đoán có thể được xem là hai bài toán tách biệt. Một ưu điểm lớn của việc tách rời chúng là cùng một bộ mã hóa có thể được dùng lại cho nhiều ứng dụng với các bộ dự đoán khác nhau. Điều này cũng thúc đẩy một hướng nghiên cứu rất hứa hẹn, trong đó một bộ mã hóa đa dụng được tiền huấn luyện (*pre-training*) trên dữ liệu quy

[1] Mặc dù chúng tôi giới hạn thảo luận ở chuỗi từ, các phương pháp này có thể xử lý chuỗi gồm bất kỳ đơn vị ngôn ngữ nào, chẳng hạn từ con (subword) hay ký tự.

[2] Kiến trúc này có thể mở rộng cho các bộ mã hóa có đầu vào và đầu ra khác độ dài. Ví dụ, đầu vào là $w_1 \cdots w_m$ còn đầu ra là $\mathbf{h}_1 \cdots \mathbf{h}_n$ $(m \neq n)$.

mô lớn rồi được dùng làm thành phần cốt lõi trong nhiều hệ thống ứng dụng phía sau [Peters và cộng sự, 2018; Devlin và cộng sự, 2019].



(a) Encoding the sequence as a vector (b) Encoding the sequence as a vector sequence

Hình 4.1: Biểu diễn một chuỗi từ dưới dạng (a) một vectơ hoặc (b) một chuỗi các vectơ. (Nhãn trong hình: *Encoder* = bộ mã hóa; *a single vector* = một vectơ duy nhất; (a) mã hóa chuỗi thành một vectơ; (b) mã hóa chuỗi thành một chuỗi vectơ.)

Chú giải của người dịch: Mã hóa rồi dự đoán: một ví dụ đời thường

Hãy hình dung việc chấm một bài văn. Bước *mã hóa* giống như giáo viên đọc bài và ghi tóm tắt các ý chính vào một phiếu nhận xét (đó là $\mathbf{H}$). Bước *dự đoán* là nhìn vào phiếu đó để cho điểm (đó là $\mathbf{y}$). Cùng một phiếu nhận xét có thể dùng cho nhiều mục đích khác nhau: cho điểm, xếp loại, hay chọn bài đăng báo trường — chỉ cần thay "người dự đoán". Đó chính là lợi ích của việc tách hai bước mà tác giả nói tới. Với Hình 4.1(b), "phiếu nhận xét" chi tiết hơn: mỗi từ w_i có riêng một vectơ $\mathbf{h}_i$ mô tả từ đó *trong bối cảnh cả câu*. Ký hiệu $\arg\max_y \Pr(y|\mathbf{H})$ nghĩa là "giá trị y làm cho $\Pr(y|\mathbf{H})$ lớn nhất". Ví dụ, nếu $\Pr(\text{Food}|\mathbf{H}) = 0.8$ và $\Pr(\text{Not-food}|\mathbf{H}) = 0.2$ thì $\hat{y} = \text{Food}$.

Có nhiều cách hiện thực $\text{Encode}(\cdot)$ và $\text{Predict}(\cdot)$. Ví dụ, trong phân loại văn bản, một cách là định nghĩa $\text{Encode}(\cdot)$ là một hàm tính vectơ đặc trưng bằng các mẫu (*template*) do con người thiết kế, còn $\text{Predict}(\cdot)$ là một bộ phân loại (*classifier*) thống kê truyền thống (ví dụ: SVM hoặc mô hình entropy cực đại). Một cách khác là định nghĩa $\text{Encode}(\cdot)$ là một mạng nơ-ron nhiều tầng cho đầu ra là một vectơ số thực, và định nghĩa $\text{Predict}(\cdot)$ là một mạng nơ-ron đơn giản chỉ gồm một tầng Softmax. Trong chương này, chúng ta tập trung vào các bộ mã hóa dựa trên mạng nơ-ron. Mục 4.5 sẽ minh họa cách áp dụng các bộ mã hóa này cho nhiều tác vụ NLP khác

nhau.

4.2 Các mô hình hồi quy

Nghiên cứu một cách toàn diện các mô hình chuỗi khác nhau không phải việc dễ dàng. Tuy nhiên, sẽ thuận tiện nếu trước tiên ta giới thiệu một trong những mô hình nơ-ron phổ biến và thực dụng nhất, gọi là mạng nơ-ron hồi quy (*recurrent neural networks, RNNs*). Về sau chúng ta sẽ thấy RNN được dùng rộng rãi trong mô hình hóa chuỗi, và các kỹ thuật trình bày ở đây mang tính tổng quát, áp dụng được cho nhiều hệ thống.

4.2.1 Mô hình ngôn ngữ dựa trên RNN

Có lẽ ứng dụng phổ biến nhất của các mô hình chuỗi trong NLP là ước lượng xác suất của một chuỗi từ, gọi là mô hình hóa ngôn ngữ (*language modeling*). Về mặt toán học, mô hình hóa ngôn ngữ là một trường hợp cụ thể của một bài toán cơ bản trong lĩnh vực **quá trình ngẫu nhiên** (*stochastic processes*, hay ***random processes***): mô hình hóa dữ liệu **chuỗi thời gian** (*time series*) [Hamilton, 1994; Chatfield, 2003; Fuller, 2009]. Khi được xem như một chuỗi thời gian, một chuỗi từ tạo thành các điểm dữ liệu rời rạc xuất hiện tại các bước thời gian liên tiếp. Theo nghĩa này, các phương pháp trình bày ở đây có thể áp dụng rộng rãi, mặc dù việc thảo luận đầy đủ về phân tích chuỗi thời gian tổng quát nằm ngoài phạm vi cuốn sách này.

Cho một chuỗi từ $w_1 \cdots w_m$, mục tiêu của mô hình hóa ngôn ngữ là tính $\Pr(w_1, \ldots, w_m)$. Xác suất đồng thời (*joint probability*) này thường được viết thành tích của các xác suất có điều kiện (*conditional probability*) theo quy tắc chuỗi:

$$\Pr(w_1, \ldots, w_m) \;=\; \Pr(w_1) \cdot \Pr(w_2|w_1) \cdots \Pr(w_m|w_1, \ldots, w_{m-1}). \tag{4.3}$$

Chú giải của người dịch: Quy tắc chuỗi của xác suất

$\Pr(B|A)$ đọc là "xác suất của B *khi biết* A đã xảy ra". Quy tắc nhân quen thuộc $\Pr(A, B) \;=\; \Pr(A) \cdot \Pr(B|A)$ áp dụng lặp lại cho cả chuỗi sẽ cho

Phương trình (4.3). Ví dụ với câu ba từ "tôi ăn cơm":

$$\Pr(\text{tôi, ăn, cơm}) = \Pr(\text{tôi}) \cdot \Pr(\text{ăn}|\text{tôi}) \cdot \Pr(\text{cơm}|\text{tôi, ăn}).$$

Nếu ba thừa số lần lượt là 0.1, 0.2 và 0.3 thì xác suất của cả câu là $0.1 \times 0.2 \times 0.3 = 0.006$. Như vậy, để tính xác suất của cả câu, ta chỉ cần biết cách *đoán từ tiếp theo khi đã biết các từ trước nó*.

Nói cách khác, bài toán sinh ra $w_1 \cdots w_m$ cũng chính là bài toán mỗi lần sinh ra một từ w_{i+1} dựa trên các từ trước đó $w_1 \cdots w_i$. Các mô hình ngôn ngữ dựa trên RNN biểu diễn $w_1 \cdots w_i$ bằng một đơn vị hồi quy $\text{RNN}(\cdot)$ [Mikolov và cộng sự, 2010]:

$$\mathbf{h}_i \;=\; \text{RNN}(\mathbf{h}_{i-1}, \mathbf{x}_i), \tag{4.4}$$

trong đó $\mathbf{x}_i \in \mathbb{R}^{d_e}$ là vectơ từ (hay vectơ nhúng từ) của w_i. Gọi V là bộ từ vựng (*vocabulary*) mà từ đó ta có thể chọn một từ. Nếu $\mathbf{1}_{w_i}$ là biểu diễn one-hot của w_i thì $\mathbf{x}_i$ được tính bằng cách nhân $\mathbf{1}_{w_i}$ với bảng nhúng từ $\mathbf{C} \in \mathbb{R}^{|V| \times d_e}$:

$$\begin{aligned} \mathbf{x}_i &= \text{Embed}(w_i) \\ &= \mathbf{1}_{w_i}\mathbf{C}. \end{aligned} \tag{4.5}$$

Như đã trình bày ở Chương 1, việc dùng $\mathbf{C}$ biến đổi một vectơ $|V|$ chiều (và có lẽ là rất nhiều chiều) thành một vectơ d_e chiều (và có lẽ là ít chiều). Lưu ý rằng $\mathbf{C}$ về bản chất là một bảng tra cứu, với mỗi từ trong V có một mục riêng (tức là một hàng) trong bảng. Vì vậy, vế phải của Phương trình (4.5) là một phép tra cứu, lấy ra hàng tương ứng của $\mathbf{C}$.

Chú giải của người dịch: Vì sao nhân với vectơ one-hot lại là "tra bảng"?

Vectơ one-hot $\mathbf{1}_{w_i}$ có $|V|$ phần tử, toàn số 0 trừ một số 1 ở vị trí của từ w_i. Giả sử bộ từ vựng chỉ có 3 từ và $d_e = 2$:

$$\mathbf{C} = \begin{bmatrix} 0.1 & 0.5 \\ 0.7 & -0.2 \\ 0.3 & 0.9 \end{bmatrix}, \qquad \mathbf{1}_w = \begin{bmatrix} 0 & 1 & 0 \end{bmatrix}.$$

Phép nhân (hàng nhân cột) cho $\mathbf{1}_w\mathbf{C} = [\,0 \cdot 0.1 + 1 \cdot 0.7 + 0 \cdot 0.3, \quad 0 \cdot 0.5 + 1 \cdot (-0.2) + 0 \cdot 0.9\,] = [0.7 \;\; -0.2]$, đúng bằng hàng thứ 2 của $\mathbf{C}$. Vì

vậy trong thực tế, máy tính không cần nhân thật mà chỉ việc "lấy hàng số 2 ra" — nhanh hơn nhiều. Ký hiệu $\mathbf{C} \in \mathbb{R}^{|V| \times d_e}$ nghĩa là $\mathbf{C}$ là một ma trận số thực có $|V|$ hàng và d_e cột.

Bây giờ ta quay lại Phương trình (4.4). Về mặt trực giác, phương trình này cho thấy trạng thái của phần ngữ cảnh đã thấy tới thời điểm hiện tại (tức $\mathbf{h}_i$) là một biểu diễn nào đó của sự kết hợp giữa đầu vào hiện tại (tức $\mathbf{x}_i$) và trạng thái của phần ngữ cảnh trước đó ($\mathbf{h}_{i-1}$). Nói cách khác, có thể xem đây là quá trình liên tục bổ sung thông tin của một từ mới vào một "bộ nhớ lịch sử". Một điểm tinh tế của quá trình này là nó có thể được hiện thực dễ dàng bằng cách áp dụng lặp lại Phương trình (4.4) cho tới hết chuỗi.

$\text{RNN}(\cdot)$ có thể là bất kỳ hàm nào nhận $\mathbf{h}_{i-1}$ và $\mathbf{x}_i$ rồi sinh ra một vectơ mới $\mathbf{h}_i$. RNN cơ bản (*vanilla RNN*) có dạng

$$\text{RNN}(\mathbf{h}_{i-1}, \mathbf{x}_i) \;=\; \psi(\mathbf{h}_{i-1}\mathbf{U} + \mathbf{x}_i\mathbf{V}), \tag{4.6}$$

trong đó $\psi(\cdot)$ là một hàm kích hoạt (*activation function*), chẳng hạn $\text{TanH}(\cdot)$ và $\text{Sigmoid}(\cdot)$. Kết hợp với các Phương trình (4.4) và (4.5), ta có thể định nghĩa $\mathbf{h}_i$ như một hàm của $\mathbf{h}_{i-1}$ và w_i

$$\mathbf{h}_i \;=\; \psi(\mathbf{h}_{i-1}\mathbf{U} + \mathbf{1}_{w_i}\mathbf{C}\mathbf{V}), \tag{4.7}$$

trong đó $\mathbf{U} \in \mathbb{R}^{d_h \times d_h}$, $\mathbf{V} \in \mathbb{R}^{d_e \times d_h}$ và $\mathbf{C} \in \mathbb{R}^{|V| \times d_e}$ là các tham số (*parameter*) học được của mô hình, còn d_h là một siêu tham số (*hyper-parameter*) cho biết số chiều của $\mathbf{h}_i$ và $\mathbf{h}_{i-1}$.

Chú giải của người dịch: RNN như một người đọc có trí nhớ

Hãy tưởng tượng bạn đọc câu "Hôm nay trời mưa to" từng từ một và sau mỗi từ ghi lại "ấn tượng hiện tại" vào một mẩu giấy. Mẩu giấy chính là $\mathbf{h}_i$. Khi đọc từ mới w_i, bạn kết hợp nội dung mẩu giấy cũ ($\mathbf{h}_{i-1}$, nhân với ma trận $\mathbf{U}$) với thông tin của từ mới ($\mathbf{x}_i$, nhân với $\mathbf{V}$), cộng lại, rồi "nén" qua hàm ψ (ví dụ tanh luôn cho giá trị trong khoảng $(-1; 1)$) để được mẩu giấy mới. Điểm then chốt: *cùng* các ma trận $\mathbf{U}, \mathbf{V}$ được dùng ở mọi bước, nên một mô hình có số tham số cố định có thể đọc câu dài bao nhiêu cũng được.
Ví dụ một chiều ($d_h = d_e = 1$), với $U = 0.5$, $V = 1$, $\psi = \tanh$, $h_0 = 0$ và các đầu vào $x_1 = 1, x_2 = -1$: $h_1 = \tanh(0.5 \cdot 0 + 1 \cdot 1) = \tanh(1) \approx 0.76$; $h_2 = \tanh(0.5 \cdot 0.76 + 1 \cdot (-1)) = \tanh(-0.62) \approx -0.55$. Giá trị h_2 vẫn

"nhớ" một phần x_1 thông qua h_1.

Giờ đây ta đã có một bộ mã hóa biểu diễn chuỗi từ $w_1 \cdots w_m$ dưới dạng một chuỗi các đầu ra của RNN $\mathbf{H} = \{\mathbf{h}_1, \ldots, \mathbf{h}_m\}$. Vì mỗi $\mathbf{h}_i$ mã hóa chuỗi con từ w_1 tới w_i, ta có thể đặt một tầng Softmax lên trên $\mathbf{h}_i$ để thu được một phân phối trên các từ:

$$\mathbf{y}_{i+1} \;=\; \text{Softmax}(\mathbf{h}_i\mathbf{O} + \mathbf{b}), \tag{4.8}$$

trong đó $\mathbf{O} \in \mathbb{R}^{d_h \times |V|}$ và $\mathbf{b} \in \mathbb{R}^{|V|}$. Lấy chỉ số của từ w_{i+1}, ta có

$$\Pr(w_{i+1}|w_1, \ldots, w_i) \;=\; y_{i+1}(w_{i+1}). \tag{4.9}$$

Như vậy, ta đã xây dựng được một mô hình ngôn ngữ sinh ra xác suất $\Pr(w_{i+1}|w_1, \ldots, w_i)$ tại mỗi bước. Hình 4.2 minh họa mô hình ngôn ngữ dựa trên RNN cho một chuỗi ví dụ. Để chạy mô hình này trên một chuỗi từ, tất nhiên ta muốn bắt đầu bằng việc dự đoán w_1, nhưng điều này đòi hỏi một từ đứng trước w_0 làm đầu vào. Một cách đơn giản và áp dụng rộng rãi để tạo trạng thái khởi đầu phù hợp cho RNN là thêm một ký hiệu bắt đầu $\langle\text{SOS}\rangle$ vào đầu chuỗi, để mọi chuỗi đều bắt đầu bằng cùng một "từ". Tương tự, ta có thể gắn một ký hiệu kết thúc $\langle\text{EOS}\rangle$ vào cuối chuỗi để mô hình hóa tính trọn vẹn của chuỗi. Điều này dẫn tới một dạng mới của xác suất của chuỗi

$$\begin{aligned}\Pr(\langle\text{SOS}\rangle, w_1, \ldots, w_m, \langle\text{EOS}\rangle) \;=\; & \Pr(\langle\text{SOS}\rangle)\cdot \\ & \Pr(w_1|\langle\text{SOS}\rangle)\cdot \\ & \Pr(w_2|\langle\text{SOS}\rangle, w_1)\cdot \\ & \ldots \\ & \Pr(w_m|\langle\text{SOS}\rangle, w_1, \ldots, w_{m-1})\cdot \\ & \Pr(\langle\text{EOS}\rangle|\langle\text{SOS}\rangle, w_1, \ldots, w_m).\end{aligned} \tag{4.10}$$

Ta có thể đơn giản giả sử $\Pr(\langle\text{SOS}\rangle) = 1$. Để tính $\Pr(\langle\text{SOS}\rangle, w_1, \ldots, w_m, \langle\text{EOS}\rangle)$, ta lấy $\langle\text{SOS}\rangle\, w_1 \cdots w_m$ làm chuỗi đầu vào và $w_1 \cdots w_m \,\langle\text{EOS}\rangle$ làm chuỗi đầu ra.

Chú giải của người dịch: Softmax và ký hiệu đầu/cuối câu

Tầng Softmax biến một vectơ điểm số bất kỳ thành một phân phối xác suất (các số dương, tổng bằng 1). Vectơ $\mathbf{y}_{i+1}$ có $|V|$ phần tử, mỗi phần tử là xác suất để *một từ* trong bộ từ vựng là từ tiếp theo; $y_{i+1}(w_{i+1})$ là phần tử ứng với đúng từ w_{i+1}. Ký hiệu ⟨SOS⟩ (*start of sequence*) và ⟨EOS⟩ (*end of sequence*) giống như "tiếng còi bắt đầu" và "dấu chấm hết": nhờ ⟨EOS⟩, mô hình học được cả việc *khi nào nên dừng* câu. Ví dụ, câu "tôi ăn cơm" cho đầu vào ⟨SOS⟩ tôi ăn cơm và đầu ra cần đoán là tôi ăn cơm ⟨EOS⟩ (lệch nhau một vị trí).

4.2.2 Huấn luyện

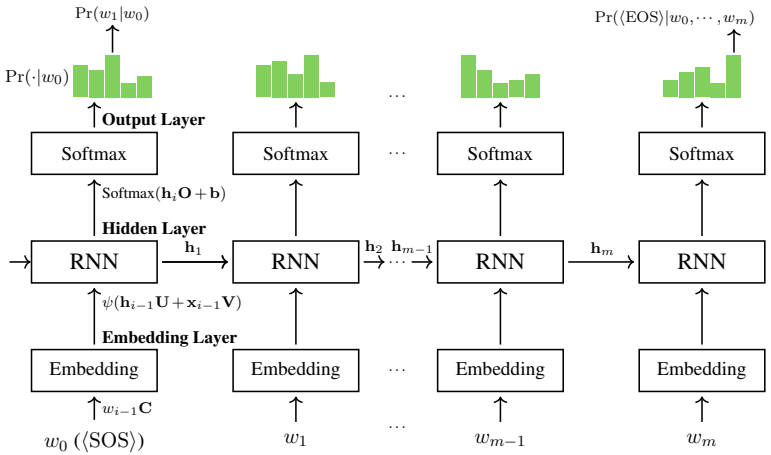
Là một kiến trúc nơ-ron khả vi, mô hình ngôn ngữ dựa trên RNN có thể được huấn luyện (*training*) bằng các kỹ thuật tối ưu hóa chuẩn. Những nguyên lý nền tảng của việc huấn luyện mạng nơ-ron đã được thảo luận kỹ ở Chương 2. Vì vậy, thay vì mô tả đầy đủ ở đây, chúng tôi chỉ phác họa các khái niệm cơ bản và thảo luận một số cải tiến dành cho các mô hình chuỗi.

Mô hình hóa ngôn ngữ dựa trên RNN có thể được phát biểu thành một bài toán dự đoán bước tiếp theo. Giả sử ta có một kho ngữ liệu (*corpus*) $\mathcal{D}$ gồm các chuỗi từ. Với mỗi chuỗi $\mathbf{w} = w_1, \ldots, w_{|\mathbf{w}|}$ trong $\mathcal{D}$, ta xây dựng một chuỗi các cặp đầu vào–đích như sau[3]:

$$\{(w_1, w_2), (w_2, w_3), \ldots, (w_{|\mathbf{w}|-1}, w_{|\mathbf{w}|})\}.$$

Mô hình ngôn ngữ xử lý chuỗi tiền tố đầu vào $w_1, \ldots, w_{|\mathbf{w}|-1}$ và đưa ra một chuỗi tương ứng các phân phối xác suất dự đoán $\hat{\mathbf{y}}_2, \ldots, \hat{\mathbf{y}}_{|\mathbf{w}|}$. Xem bảng sau để thấy minh họa đầu vào và đầu ra của mô hình.

[3] Mặc dù các token biên ⟨SOS⟩ và ⟨EOS⟩ là thiết yếu trong các hiện thực thực tế, ở đây chúng tôi lược bỏ chúng khỏi ký hiệu cho đơn giản.



Hình 4.2: Minh họa việc dùng mô hình ngôn ngữ dựa trên RNN để tính $\Pr(\langle\text{SOS}\rangle w_1 \cdots w_m \langle\text{EOS}\rangle)$. Đầu vào là $\langle\text{SOS}\rangle w_1 \cdots w_m$, còn đầu ra là xác suất $\Pr(w_1|\langle\text{SOS}\rangle)\Pr(w_2|\langle\text{SOS}\rangle\, w_1)\cdots\Pr(\langle\text{EOS}\rangle|\langle\text{SOS}\rangle\, w_1 \cdots w_m)$. Vì $\Pr(\langle\text{SOS}\rangle) = 1$, xác suất sinh ra chuỗi đơn giản là $\Pr(\langle\text{SOS}\rangle w_1 \cdots w_m \langle\text{EOS}\rangle) = \Pr(\langle\text{SOS}\rangle)\Pr(w_1|\langle\text{SOS}\rangle)\Pr(w_2|\langle\text{SOS}\rangle\, w_1)\cdots\Pr(\langle\text{EOS}\rangle|\langle\text{SOS}\rangle w_1 \cdots w_m)$. Với mỗi đầu vào w_i, trước tiên ta biểu diễn nó thành một vectơ từ $\mathbf{x}_i$ qua tầng nhúng, thu được một chuỗi vectơ từ $\mathbf{x}_0 \cdots \mathbf{x}_m$. Tầng RNN ánh xạ $\mathbf{x}_0 \cdots \mathbf{x}_m$ thành một chuỗi các trạng thái ẩn (*hidden state*) $\mathbf{h}_1 \cdots \mathbf{h}_{m+1}$. Trong quá trình này, phép toán được áp dụng một cách đệ quy: một đơn vị RNN nhận cả $\mathbf{h}_{i-1}$ và $\mathbf{x}_{i-1}$ rồi sinh ra trạng thái mới $\mathbf{h}_i$. Bên trên đó, ta dùng tầng đầu ra (Softmax) để thu được $\Pr(w_{i+1}|\langle\text{SOS}\rangle\, w_1 \cdots w_i)$. (Nhãn trong hình: *Embedding Layer* = tầng nhúng; *Hidden Layer* = tầng ẩn; *Output Layer* = tầng đầu ra.)

Bước	Lịch sử	Đầu vào	Đầu ra (phân phối)	Đáp án chuẩn
1		w_1	$\hat{\mathbf{y}}_2$	w_2
2	w_1	w_2	$\hat{\mathbf{y}}_3$	w_3
3	w_1, w_2	w_3	$\hat{\mathbf{y}}_4$	w_4
...	...	...	...	...
$\lvert\mathbf{w}\rvert - 2$	$w_1, w_2, \ldots, w_{\lvert\mathbf{w}\rvert-3}$	$w_{\lvert\mathbf{w}\rvert-2}$	$\hat{\mathbf{y}}_{\lvert\mathbf{w}\rvert-1}$	$w_{\lvert\mathbf{w}\rvert-1}$
$\lvert\mathbf{w}\rvert - 1$	$w_1, w_2, \ldots, w_{\lvert\mathbf{w}\rvert-3}, w_{\lvert\mathbf{w}\rvert-2}$	$w_{\lvert\mathbf{w}\rvert-1}$	$\hat{\mathbf{y}}_{\lvert\mathbf{w}\rvert}$	$w_{\lvert\mathbf{w}\rvert}$

Một hàm mất mát (*loss function*) $\mathcal{L}(\hat{\mathbf{y}}_i, \mathbf{1}_{w_i})$ được định nghĩa để đo mức chênh lệch giữa phân phối dự đoán $\hat{\mathbf{y}}_i$ và biểu diễn one-hot của đích thực $\mathbf{1}_{w_i}$. Lựa chọn tiêu chuẩn cho tác vụ này là hàm mất mát entropy chéo (*cross-entropy*), đo độ sai khác giữa hai phân phối [Mitchell, 1997; Bishop, 2006]. Khi đó, mất mát trên toàn bộ tập dữ liệu được định nghĩa là:

$$\mathcal{L} \;=\; \sum_{\mathbf{w}\in\mathcal{D}} \sum_{i=2}^{|\mathbf{w}|} \mathcal{L}(\hat{\mathbf{y}}_i, \mathbf{1}_{w_i}). \tag{4.11}$$

Chú giải của người dịch: Entropy chéo với vectơ one-hot

Khi đích là vectơ one-hot, entropy chéo rút gọn thành $\mathcal{L}(\hat{\mathbf{y}}_i, \mathbf{1}_{w_i}) = -\log \hat{y}_i(w_i)$, tức là "trừ logarit của xác suất mà mô hình gán cho từ đúng". Nếu mô hình gán xác suất 0.9 cho từ đúng thì mất mát là $-\ln 0.9 \approx 0.105$ (nhỏ, tốt); nếu chỉ gán 0.01 thì mất mát là $-\ln 0.01 \approx 4.6$ (lớn, tệ). Phương trình (4.11) chỉ đơn giản cộng các mất mát này qua mọi vị trí i của mọi câu $\mathbf{w}$ trong kho ngữ liệu. Tổng bắt đầu từ $i = 2$ vì từ đầu tiên w_1 không được dự đoán (xem bảng trên).

Khi đã biết mất mát, việc huấn luyện mô hình ngôn ngữ dựa trên RNN có thể thực hiện bằng hạ gradient (*gradient descent*). Một dạng đơn giản của phương pháp này là quy tắc delta (*delta rule*)

$$\theta_{\text{new}} \;=\; \theta_{\text{old}} - \eta \cdot \frac{\partial \mathcal{L}}{\partial \theta}, \tag{4.12}$$

trong đó θ ký hiệu các tham số. Với mô hình mô tả ở Mục 4.2.1, θ gồm $\mathbf{C}$, $\mathbf{U}$, $\mathbf{V}$, $\mathbf{O}$ và $\mathbf{b}$. $\frac{\partial \mathcal{L}}{\partial \theta}$ là đạo hàm của mất mát theo các tham số, gọi là **gradient lỗi** (*error gradient*).

Phương trình (4.12) có thể được hiểu là quá trình dịch chuyển các tham số hiện tại một bước nhỏ theo hướng dốc xuống nhanh nhất (tức là hướng

của $-\frac{\partial \mathcal{L}}{\partial \theta}$). Ở đây η biểu thị ta đi xa bao nhiêu trong mỗi bước xuống dốc, còn được gọi là tốc độ học (*learning rate*). Để tính $\frac{\partial \mathcal{L}}{\partial \theta}$ thường cần một thuật toán lan truyền ngược (*back-propagation*) truyền lỗi ngược qua các bước thời gian đã được trải ra. Trong các hiện thực hiện đại của hệ thống học sâu (*deep learning*), nơi mạng nơ-ron được biểu diễn dưới dạng đồ thị tính toán (*computation graph*), lan truyền ngược trở nên đơn giản vì nó chỉ là sản phẩm phụ của việc duyệt đồ thị, và có nhiều bộ công cụ vi phân tự động làm việc này. Các thuật toán tương tự, gọi là **lan truyền ngược theo thời gian** (*back-propagation through time, BPTT*), cũng đã được dùng trong các hệ thống trước đây [Werbos, 1990]. Để tìm hiểu thêm về huấn luyện mạng nơ-ron, xem Chương 2 và/hoặc các giáo trình về chủ đề này [Goodfellow và cộng sự, 2016; Zhang và cộng sự, 2021a].

Chú giải của người dịch: Hạ gradient: xuống núi trong sương mù

Hãy hình dung $\mathcal{L}$ là độ cao của một ngọn đồi, còn θ là vị trí của bạn. Bạn muốn xuống thung lũng (mất mát nhỏ nhất) nhưng sương mù dày đặc, chỉ cảm nhận được độ dốc ngay dưới chân. Đạo hàm $\frac{\partial \mathcal{L}}{\partial \theta}$ cho biết độ dốc; đi ngược dấu đạo hàm là đi xuống. Ví dụ một tham số: $\mathcal{L}(\theta) = \theta^2$, $\theta_{\text{old}} = 3$, $\eta = 0.1$. Đạo hàm là $2\theta = 6$, nên $\theta_{\text{new}} = 3 - 0.1 \cdot 6 = 2.4$, gần cực tiểu $\theta = 0$ hơn. Khi có nhiều tham số, $\frac{\partial \mathcal{L}}{\partial \theta}$ là tập hợp các *đạo hàm riêng* theo từng tham số (giữ các tham số khác cố định), gọi chung là gradient. "Trải ra theo thời gian" nghĩa là vẽ RNN thành chuỗi các bản sao nối tiếp nhau như Hình 4.2, rồi lan truyền ngược qua chuỗi đó.

Nếu đầu vào là một chuỗi dài, việc áp dụng RNN sẽ tạo ra một mạng nơ-ron rất sâu. Khi đó, việc dùng quy tắc chuỗi cho đạo hàm có thứ tự khiến các đạo hàm của mất mát rất lớn hoặc rất nhỏ bị tích lũy lại, và do đó bước cập nhật tham số trong Phương trình (4.12) trở nên rất lớn hoặc rất nhỏ. Hiện tượng này thường được gọi là **bài toán bùng nổ gradient và tiêu biến gradient** (*exploding and vanishing gradient problems*). Có một số phương pháp để giảm nhẹ các vấn đề này cho RNN [Sutskever, 2013]. Trong đó có:

- **Chính quy hóa** (*Regularization*). Đưa các số hạng chính quy hóa (như chuẩn l_1 và l_2 trên các ma trận tham số) vào quá trình huấn luyện có thể tránh được các mô hình mà phần lớn tham số có giá trị lớn, và nhờ vậy giúp tránh bùng nổ gradient. Tương tự, ta có thể

phạt các trường hợp chuẩn của gradient quá nhỏ [Pascanu và cộng sự, 2013].

- **Cắt gradient** (*Gradient Clipping*). Khi chuẩn của gradient quá lớn, một cách tự nhiên là trực tiếp thu nhỏ độ lớn của chúng. Một phương pháp đơn giản là cắt chuẩn gradient theo một ngưỡng τ. Nếu chuẩn $||\frac{\partial \mathcal{L}}{\partial \theta}||$ lớn hơn τ, ta có thể co giãn lại $\frac{\partial \mathcal{L}}{\partial \theta}$ cho phù hợp, ví dụ $\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\tau}{||\frac{\partial \mathcal{L}}{\partial \theta}||} \cdot \frac{\partial \mathcal{L}}{\partial \theta}$[4].

- **Lan truyền ngược bị cắt ngắn** (*Truncated Back-propagation*). Một ý tưởng khác là chia một chuỗi dài các cặp đầu vào–đầu ra thành các đoạn ngắn hơn, rồi huấn luyện RNN trên các chuỗi con riêng biệt này [Williams và Peng, 1990; Elman, 1990]. Cách này giảm cả chi phí huấn luyện lẫn nguy cơ gradient lỗi tích lũy có giá trị quá lớn hoặc quá nhỏ.

- **Cải tiến kiến trúc** (*Improved Architectures*). Cũng có thể thiết kế lại mô hình để vượt qua các giới hạn của RNN tiêu chuẩn, thường bằng cơ chế bộ nhớ. Ở Mục 4.3, ta sẽ thấy một vài ví dụ thiết kế lại đơn vị hồi quy để giải quyết vấn đề tiêu biến gradient.

- **Khởi tạo và ràng buộc tham số** (*Initialization and Constraints of Parameters*). Khởi tạo các tham số của mô hình trong một vùng thích hợp nói chung có ích cho việc tối ưu hóa, và đôi khi giúp ngăn gradient quá nhỏ. Một phương pháp khác là đặt mô hình một cách ngẫu nhiên và chỉ học tham số của các tầng đầu ra [Jaeger và Haas, 2004].

- **Hàm kích hoạt không bão hòa** (*Non-saturating Activations*). Nhiều hàm kích hoạt phổ biến có miền giá trị đầu ra bị chặn, ví dụ hàm Sigmoid có miền giá trị $[0, 1]$. Chúng còn được gọi là **hàm kích hoạt bão hòa** (*saturating activation functions*)[5]. Việc dùng hàm kích hoạt bão hòa thường dẫn tới gradient suy giảm dần qua

[4]Nó thường được phát biểu dưới dạng phương trình

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\tau}{\max(\tau, ||\frac{\partial \mathcal{L}}{\partial \theta}||)} \cdot \frac{\partial \mathcal{L}}{\partial \theta}. \tag{4.13}$$

[5]Một hàm kích hoạt $f(x)$ là không bão hòa khi và chỉ khi khi $x \to \infty$ (hoặc $-\infty$) thì $f(x) \to \infty$. Một hàm kích hoạt là bão hòa nếu nó không phải hàm kích hoạt không bão hòa.

các tầng, tức là tiêu biến gradient. Do đó, dùng hàm kích hoạt không bão hòa thay thế, ví dụ hàm ReLU, là một hướng đầy hứa hẹn.

- **Chuẩn hóa giá trị kích hoạt** (*Normalization of Activations*). Hàm kích hoạt bão hòa còn có thể khiến mô hình bị mắc kẹt trong một vùng bão hòa của đầu ra, và ta cần tốc độ học lớn để thoát khỏi các cực trị địa phương [Ioffe và Szegedy, 2015]. Khi đó, quá trình huấn luyện sẽ không ổn định, và những thay đổi nhỏ ở đầu vào và/hoặc tham số mô hình có thể dẫn tới biến động lớn trong hành vi của mô hình. Một giải pháp khả dĩ là chuẩn hóa các giá trị kích hoạt để giảm phương sai, ví dụ trừ đi trung bình của các giá trị kích hoạt trong một nhóm mẫu (chẳng hạn các mẫu trong một lô nhỏ (*mini-batch*) huấn luyện) rồi chia cho độ lệch chuẩn của chúng.

Chú giải của người dịch: Vì sao gradient "bùng nổ" hay "tiêu biến"?

Theo quy tắc chuỗi, gradient truyền ngược qua n bước của RNN gần giống một *tích* của n thừa số (mỗi bước một thừa số). Tích của nhiều số lặp lại rất nhạy: $0.9^{50} \approx 0.005$ (gần như biến mất), còn $1.1^{50} \approx 117$ (phình to). Vì thế với câu dài, tín hiệu học từ các từ ở xa hoặc gần như mất hẳn (tiêu biến), hoặc lớn tới mức làm tham số nhảy loạn (bùng nổ).
Chuẩn $||\mathbf{g}||$ của một vectơ là độ dài của nó, như độ dài vectơ ở lớp 10: $||(3,4)|| = \sqrt{3^2+4^2} = 5$. Cắt gradient với $\tau = 1$ sẽ biến gradient $(3,4)$ thành $\frac{1}{5} \cdot (3,4) = (0.6, 0.8)$: giữ nguyên hướng nhưng độ dài chỉ còn 1. Phương trình (4.13) dùng max để gradient có chuẩn nhỏ hơn τ được giữ nguyên. Hàm bão hòa như Sigmoid "phẳng lì" ở hai đầu nên đạo hàm ở đó gần 0; còn $\text{ReLU}(x) = \max(0, x)$ có đạo hàm bằng 1 với mọi $x > 0$.

4.2.3 Xếp chồng tầng

Nếu xem việc áp dụng một đơn vị hồi quy như một hàm ánh xạ một chuỗi biến thành một chuỗi biến mới có cùng độ dài, thì việc hợp hàm này với một hàm khác cùng loại, hoặc thậm chí với chính nó, là điều tự nhiên. Nhờ vậy, ta có thể dễ dàng mở rộng RNN thành mạng nơ-ron sâu chỉ bằng cách xếp chồng các tầng hồi quy.

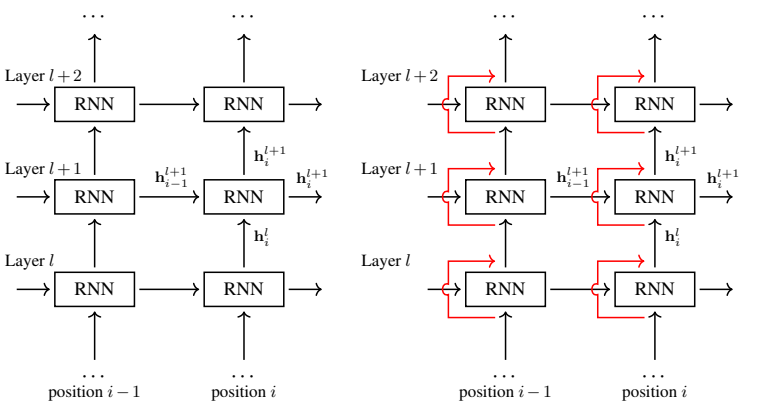
Gọi $\mathbf{h}_i^l$ là đầu ra của đơn vị hồi quy thứ l trong chồng tại vị trí i. Ta

có thể áp dụng một đơn vị hồi quy mới lên $\mathbf{h}_i^l$, thu được một đầu ra mới ở mức $l+1$

$$\mathbf{h}_i^{l+1} = \text{RNN}(\mathbf{h}_{i-1}^{l+1}, \mathbf{h}_i^l), \tag{4.14}$$

trong đó $\mathbf{h}_{i-1}^{l+1}$ là đầu ra của bước trước ở mức $l+1$. Để Phương trình (4.14) được xác định đầy đủ, ta thường định nghĩa $\mathbf{h}_i^0 = \mathbf{x}_i$. Nói cách khác, chồng tầng bắt đầu từ vectơ từ $\mathbf{x}_i$, sau đó là một loạt đầu ra RNN (tức là $\mathbf{h}_i^1$, $\mathbf{h}_i^2$, $\mathbf{h}_i^3$, v.v.).

Để minh họa, Hình 4.3(a) cho thấy một RNN xếp chồng dùng cho mô hình hóa ngôn ngữ. Ta thấy rằng áp dụng một chồng các đơn vị hồi quy tương đương với việc tạo đồng thời nhiều tầng RNN. Tuy nhiên, có thể gây nhầm lẫn nếu ta gọi một mạng hồi quy đã trải ra là một *tầng*, vì thuật ngữ *tầng* thường chỉ một tập các nơ-ron nhận cùng đầu vào trong một mạng nơ-ron truyền thẳng. Ở đây ta mở rộng thuật ngữ *tầng* cho một khái niệm tổng quát hơn: một nhóm nơ-ron được đặt ở cùng một mức về mặt cấu trúc liên kết. Vì vậy, ta nói mô hình ngôn ngữ trong Hình 4.3 có 3 tầng RNN.



(a) A 3-layer RNN. (b) A 3-layer RNN with residual connections.

Hình 4.3: RNN 3 tầng (có và không có kết nối tắt). Để xếp chồng các tầng RNN, ta đưa đầu ra của tầng l vào tầng $l+1$. Do đó đầu ra của tầng $l+1$ là $\mathbf{h}_i^{l+1} = \text{RNN}(\mathbf{h}_{i-1}^{l+1}, \mathbf{h}_i^l)$. Các đường màu đỏ biểu diễn kết nối tắt, cộng trực tiếp đầu vào của một tầng vào đầu ra của nó, cho ta $\mathbf{h}_i^{l+1} = \text{RNN}(\mathbf{h}_{i-1}^{l+1}, \mathbf{h}_i^l) + \mathbf{h}_i^l$. (Nhãn trong hình: *Layer* = tầng; *position* = vị trí; (a) RNN 3 tầng; (b) RNN 3 tầng có kết nối tắt.)

Khi xếp chồng nhiều tầng RNN, ta xây dựng được một mô hình sâu hơn

nhưng khó huấn luyện hơn. Khó khăn này một phần đến từ trở ngại trong việc truyền thông tin qua RNN nhiều tầng. Để việc huấn luyện dễ dàng hơn, một cách tiếp cận phổ biến là đưa **kết nối nhảy** (*skip connections*) hay **kết nối tắt** (*residual connections*) vào mạng nơ-ron nhiều tầng [He và cộng sự, 2016a]. Các kết nối này nhằm tận dụng một đường đi bổ sung để thông tin có thể bỏ qua một số tầng. Như đã mô tả ở Chương 2, dạng của một mạng nơ-ron residual được cho bởi:

$$\mathbf{y}^{l+1} \;=\; F(\mathbf{y}^{l}) + \mathbf{y}^{l}, \tag{4.15}$$

trong đó $\mathbf{y}^l$ là đầu ra của tầng l. Mở rộng công thức này cho Phương trình (4.14) ta được RNN nhiều tầng có kết nối tắt:

$$\mathbf{h}_i^{l+1} \;=\; \text{RNN}(\mathbf{h}_{i-1}^{l+1}, \mathbf{h}_i^{l}) + \mathbf{h}_i^{l}. \tag{4.16}$$

Điểm khác biệt duy nhất so với Phương trình (4.14) là ta thêm ánh xạ đồng nhất của $\mathbf{h}_i^l$ vào vế phải của Phương trình (4.16). Nhờ đó, đầu vào $\mathbf{h}_i^l$ có thể được tầng $l+1$ truy cập trực tiếp. Điều này đơn giản hóa rất nhiều cách thông tin lưu chuyển qua mạng nơ-ron, và cho phép hệ thống "bỏ qua" các tầng khi lan truyền lỗi. Hình 4.3(b) cho thấy một RNN 3 tầng có kết nối tắt.

Chú giải của người dịch: Chỉ số trên, chỉ số dưới và kết nối tắt

Trong $\mathbf{h}_i^l$, chỉ số dưới i là *vị trí* trong câu (từ thứ mấy), chỉ số trên l là *tầng* (tầng thứ mấy) — không phải số mũ. Ở Hình 4.3, i chạy theo chiều ngang, l chạy theo chiều dọc. Kết nối tắt giống như một "đường cao tốc" song song với con đường quanh co qua hàm RNN: nếu tầng $l+1$ chưa học được gì hữu ích, đầu ra vẫn ít nhất bằng $\mathbf{h}_i^l$, và gradient cũng có thể đi thẳng ngược xuống các tầng dưới qua phép cộng mà không bị thu nhỏ.

4.2.4 Các mô hình hai chiều

Việc dùng RNN cho phép ta phát biểu bài toán mã hóa một chuỗi từ thành bài toán sinh các từ từ trái sang phải. Một ưu điểm của cách tiếp cận này là việc mô hình hóa thông tin ngữ cảnh nảy sinh một cách tự nhiên: đầu ra của một đơn vị RNN theo một cách nào đó mô tả các từ ngữ cảnh tính tới thời điểm đó. Đặc điểm này khiến việc mô hình hóa phân phối xác suất

$\Pr(w_{i+1}|w_1, \ldots, w_i)$ trở nên rất trực tiếp, vì ta có thể dùng $\mathbf{h}_i$ làm biểu diễn của ngữ cảnh $w_1 \cdots w_i$, tức là $\Pr(w_{i+1}|w_1, \ldots, w_i) = \Pr(w_{i+1}|\mathbf{h}_i)$.

Việc sinh từ trái sang phải được dùng rộng rãi trong sinh chuỗi, chẳng hạn dịch máy. Nó có thể được xem là một trường hợp của **quá trình tự hồi quy** (*autoregressive processes, AR processes*), trong đó trạng thái của một biến phụ thuộc vào trạng thái của các biến trước nó [Chatfield, 2003; Box và cộng sự, 2015][6]. Tuy nhiên, phương pháp này không phải lựa chọn duy nhất để mô hình hóa chuỗi. Thậm chí ta không nhất thiết phải giới hạn ở mô hình hóa ngôn ngữ khi huấn luyện một bộ mã hóa chuỗi. Điều này dẫn tới một câu hỏi thú vị: làm thế nào để xây dựng một bộ mã hóa chuỗi từ mà không cần giả định gì về bộ dự đoán? Trả lời câu hỏi này dẫn ta tới việc tách việc học bộ mã hóa văn bản khỏi một tác vụ NLP cụ thể, và coi nó như một tác vụ riêng mà kết quả có thể áp dụng cho nhiều hệ thống khác. Thảo luận chi tiết hơn không phải trọng tâm ở đây và chúng tôi để dành cho các chương sau.

Chú giải của người dịch: Tự hồi quy: một ví dụ bằng số

Phương trình (4.17) với $p = 2$, $\alpha_1 = 0.5$, $\alpha_2 = 0.3$ cho $z_t = 0.5z_{t-1} + 0.3z_{t-2} + \epsilon_t$. Nếu $z_1 = 10$, $z_2 = 20$ và bỏ qua sai số thì $z_3 = 0.5 \cdot 20 + 0.3 \cdot 10 = 13$, $z_4 = 0.5 \cdot 13 + 0.3 \cdot 20 = 12.5$, … Mỗi giá trị mới được "đoán" từ

[6] Là một quá trình ngẫu nhiên, quá trình tự hồi quy biểu diễn một biến tại thời điểm t bằng cách liên hệ nó với các giá trị trong quá khứ của quá trình và giá trị hiện tại của một quá trình sai số [Chatfield, 2003]. Một cách hình thức, một chuỗi thời gian $\{z_1, \ldots, z_T\}$ mô tả một quá trình tự hồi quy bậc p nếu với mọi $t \in \{p+1, \ldots, T\}$

$$z_t = \sum_{i=1}^{p} \alpha_i z_{t-i} + \epsilon_t, \tag{4.17}$$

trong đó $\{\alpha_1, \ldots, \alpha_p\}$ là các tham số của quá trình, và ϵ_t là sai số tại thời điểm t. Quá trình này được gọi là *hồi quy* (*regressive*) vì nó có cùng dạng với **mô hình hồi quy tuyến tính bội** (*multiple linear regression model*). Tiền tố *tự* (*auto-*) xuất phát từ cách ta hồi quy z_t: z_t phụ thuộc vào các giá trị quá khứ của chính nó thay vì vào các biến độc lập bổ sung. Một cách diễn giải mô hình hóa ngôn ngữ dưới góc nhìn quá trình tự hồi quy là coi $\{z_1, \ldots, z_T\}$ như biểu diễn của một chuỗi từ $\{w_1, \ldots, w_T\}$. Nhờ vậy, ta có thể hình dung được phần nào việc dự đoán w_t từ các từ trước đó $\{w_1, \ldots, w_{t-1}\}$ bằng cách xét tính chất tự hồi quy của bài toán. Tuy nhiên, cần lưu ý rằng phần lớn các mô hình sinh chuỗi dùng trong NLP không tương đương về mặt toán học với Phương trình (4.17), mặc dù chúng thường được gọi là các mô hình hồi quy. Ví dụ, mô hình ngôn ngữ dựa trên RNN bàn ở đây không phải là mô hình tuyến tính. Thay vào đó, nó dùng nhiều tầng phi tuyến để mô tả các quan hệ phức tạp giữa các từ.

chính các giá trị trước đó của chuỗi — giống như đoán nhiệt độ ngày mai từ nhiệt độ hai hôm trước. Mô hình ngôn ngữ cũng "đoán từ tiếp theo từ các từ đã có", nhưng bằng một hàm phi tuyến phức tạp hơn nhiều thay vì một tổng có trọng số đơn giản.

Bây giờ ta trình bày một mở rộng đơn giản của mô hình chuỗi từ trái sang phải bằng cách quay lại với RNN. Lưu ý rằng trong mô hình hóa chuỗi, điều ta mong muốn là một biểu diễn nào đó của toàn bộ chuỗi. Một vấn đề của RNN thông thường là chúng là **mô hình một chiều** (*uni-directional models*), trong đó các từ ngữ cảnh đứng sau w_i bị bỏ qua. Để xét cả ngữ cảnh bên trái lẫn bên phải của một từ, ta có thể dùng **mô hình hai chiều** (*bi-directional models*). Hình 4.4 cho thấy một ví dụ về RNN hai chiều. Có hai mô hình con: một RNN từ trái sang phải và một RNN từ phải sang trái. Chúng có cùng kiến trúc nhưng hoạt động theo hai hướng ngược nhau. Với mỗi từ đầu vào w_i, RNN từ trái sang phải cho ra một vectơ biểu diễn ngữ cảnh $\{w_1, \ldots, w_i\}$ (ký hiệu là $\overrightarrow{\mathbf{h}}_i$), còn RNN từ phải sang trái cho ra một vectơ biểu diễn ngữ cảnh $\{w_i, \ldots, w_m\}$ (ký hiệu là $\overleftarrow{\mathbf{h}}_i$). Ta có thể ghép nối $\overrightarrow{\mathbf{h}}_i$ và $\overleftarrow{\mathbf{h}}_i$ để thu được một biểu diễn hai chiều

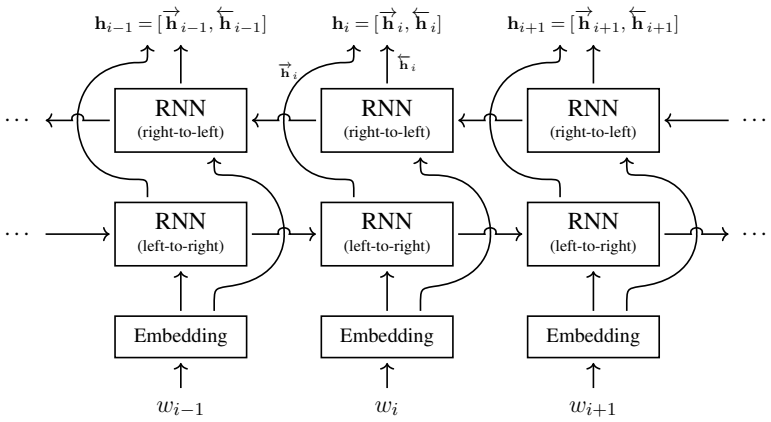
$$\mathbf{h}_i \;=\; [\overrightarrow{\mathbf{h}}_i, \overleftarrow{\mathbf{h}}_i]. \tag{4.18}$$

Như vậy, RNN hai chiều có đầu ra cùng dạng với RNN một chiều, tức là một chuỗi vectơ $\{\mathbf{h}_1, \ldots, \mathbf{h}_m\}$. Khác với RNN một chiều, biểu diễn $\mathbf{h}_i$ ở đây mô tả ngữ cảnh ở cả hai phía.

Chú giải của người dịch: Ghép nối vectơ

Phép ghép nối $[\mathbf{a}, \mathbf{b}]$ đơn giản là đặt hai vectơ nối đuôi nhau: nếu $\overrightarrow{\mathbf{h}}_i = [0.2 \;\; 0.7]$ và $\overleftarrow{\mathbf{h}}_i = [-0.1 \;\; 0.4 \;\; 0.9]$ thì $\mathbf{h}_i = [0.2 \;\; 0.7 \;\; -0.1 \;\; 0.4 \;\; 0.9]$ (5 chiều). Lợi ích của hai chiều: trong câu "Con đường này *bàn* giao cho thành phố" và "Cái *bàn* này bằng gỗ", nghĩa của "bàn" chỉ rõ khi nhìn cả các từ phía sau.

Để tăng khả năng biểu đạt, RNN hai chiều có thể được mở rộng thành mạng nơ-ron gồm nhiều tầng RNN. Ví dụ, ta có thể chạy các RNN sâu theo hai hướng rồi kết hợp kết quả như trong Phương trình (4.18). Những kiến trúc mô hình như vậy đã được dùng rộng rãi trong các tác vụ xử lý ngôn ngữ và tiếng nói, bao gồm dịch máy [Wu và cộng sự, 2016], phân tích cảm xúc [Tang và cộng sự, 2015], gán nhãn từ loại (POS) [Huang và



Hình 4.4: Một mô hình RNN hai chiều. Cho một chuỗi từ, ta chạy một RNN từ trái sang phải và một RNN khác từ phải sang trái. Do đó, tại mỗi vị trí ta thu được một biểu diễn từ trái sang phải và một biểu diễn từ phải sang trái. Đầu ra là phép ghép nối hai biểu diễn này, nhờ vậy nó bao hàm cả ngữ cảnh bên trái lẫn bên phải. (Nhãn trong hình: *left-to-right* = từ trái sang phải; *right-to-left* = từ phải sang trái; *Embedding* = tầng nhúng.)

cộng sự, 2015], nhận dạng tiếng nói (*speech recognition*) [Graves và cộng sự, 2013a;b], v.v.

4.3 Bộ nhớ

RNN phù hợp cho việc học chuỗi vì chúng tóm tắt hiệu quả các đầu vào trong quá khứ tại mỗi bước và dựa trên bản tóm tắt lịch sử này cho các dự đoán tiếp theo. Một lợi ích then chốt của RNN là ta có thể biểu diễn một lịch sử có độ dài bất kỳ bằng một vectơ kích thước cố định và cập nhật nó một cách linh động khi có thông tin mới. Nói cách khác, chúng dùng một **bộ nhớ** (*memory*) ngầm định để lưu trữ thông tin ngữ cảnh. Tiếp theo, chúng tôi sẽ chỉ ra rằng cơ chế bộ nhớ này là một khái niệm tổng quát và có thể được dùng để cải thiện các mô hình chuỗi.

4.3.1 Bộ nhớ như một hệ thống

Trong tâm lý học, trí nhớ là năng lực nhận thức của tâm trí để mã hóa, lưu trữ và truy xuất thông tin. Trong số nhiều mô hình nhận thức của tâm lý học, một khung nền tảng là **mô hình đa kho** (*multi-store model*) [Atkinson và Shiffrin, 1968]. Mô hình này định nghĩa trí nhớ là một hệ thống gồm ba thành phần cấu trúc: **trí nhớ giác quan** (*sensory memory*) thoáng qua, **trí nhớ ngắn hạn** (*short-term memory*) và **trí nhớ dài hạn** (*long-term memory*). Trí nhớ giác quan lưu giữ các kích thích thô vốn phai đi rất nhanh, chẳng hạn dữ liệu thị giác và khứu giác tức thời. Trí nhớ ngắn hạn lưu thông tin lâu hơn một chút nhưng vẫn chỉ tạm thời. Ví dụ, khi cố ghi nhớ một số điện thoại, các chữ số chỉ được giữ tạm và có thể bị quên ngay sau đó. Ngược lại, trí nhớ dài hạn là bền vững, lưu giữ thông tin vô thời hạn. Chẳng hạn, người lớn có thể nhớ lại những sự kiện cụ thể từ thời thơ ấu.

Với cách phân loại này, dường như có những liên hệ thú vị giữa mô hình trí nhớ trên và các mạng nơ-ron ta đang bàn. Ví dụ, trạng thái ẩn của một đơn vị hồi quy có thể được xem là trí nhớ ngắn hạn: nó duy trì thông tin hoạt động trong suốt một chuỗi nhưng bị đặt lại hoàn toàn khi chuyển sang một chuỗi mới[7]. Mặt khác, toàn bộ mô hình ngôn ngữ cùng các tham số của nó hoạt động giống một trí nhớ dài hạn: mô hình mã hóa các khuôn mẫu xác suất tổng quát để dự đoán từ rút ra từ kho ngữ liệu huấn luyện, và lưu giữ vĩnh viễn tri thức này cho suy luận (*inference*) về sau. Hơn nữa, các thước đo nhận thức tiêu chuẩn khớp hoàn hảo với các khái niệm tính toán: **mã hóa** (*coding* hay *encoding*) chỉ cách thông tin được lưu trong bộ nhớ, **thời lượng** (*duration*) chỉ thông tin được lưu trong bộ nhớ bao lâu, và **dung lượng** (*capacity*) chỉ bao nhiêu thông tin được lưu trong bộ nhớ.

Trong học máy và NLP, ta có thể hiểu bộ nhớ bằng cách xét nó từ góc độ xử lý thông tin. Nói chung, bộ nhớ có thể được xem là một hệ thống ghi thông tin vào một "kho lưu trữ" và đọc nó ra khi được truy vấn. Nó có các chức năng sau.

- **Mã hóa** (*Encoding*). Đầu vào của hệ thống được mã hóa dưới một dạng dễ xử lý. Với đầu vào là văn bản, có thể đơn giản xem đây là cùng quá trình mã hóa mà ta bàn ở cả Chương 3 và chương này:

[7] Hoặc, trạng thái ẩn tại bước i có thể đơn giản chỉ còn giữ rất ít thông tin về các bước đầu tiên do thông tin bị phai dần.

một từ hoặc một chuỗi từ được biểu diễn thành một vectơ đặc trưng hoặc một chuỗi vectơ đặc trưng.

- **Cập nhật** (*Update*). Với thông tin đã mã hóa, ta lưu nó vào bộ nhớ. Thao tác này nói chung phụ thuộc vào cách tổ chức bộ nhớ. Ví dụ, có thể coi một nhóm các mục đã mã hóa như một kho dữ liệu có hệ thống chỉ mục. Khi đó, lưu một mục đòi hỏi tìm đúng chỗ để đặt nó. Một cách khác là biểu diễn bộ nhớ như một vectơ số duy nhất.

- **Truy xuất** (*Retrieval*). Thông tin đã lưu có thể được truy xuất. Việc này thường gồm đối sánh từng mục trong bộ nhớ với một truy vấn đầu vào. Nếu bộ nhớ được biểu diễn ở dạng đơn giản hơn, chẳng hạn một vectơ, thì có thể không cần truy xuất tường minh, và ta trả về toàn bộ bộ nhớ khi cần.

Các chức năng này có thể được thiết kế theo nhiều cách khác nhau, dẫn tới nhiều hệ thống NLP đa dạng. Một ví dụ đơn giản là truy xuất thông tin (*information retrieval, IR*) [Manning và cộng sự, 2008]. Một hệ thống IR điển hình lập chỉ mục cho một bộ sưu tập tài liệu khổng lồ và cho phép người dùng tìm kiếm thông tin liên quan. Để tìm kiếm hiệu quả, các tài liệu được mã hóa thành những biểu diễn thuận tiện. Ví dụ, một mô hình túi từ (*bag-of-words*) có thể tính điểm khớp giữa một tài liệu và một truy vấn, trong khi một chỉ mục ngược (*inverted index*) ánh xạ hiệu quả các từ trong truy vấn tới vị trí lưu trữ của chúng. Các hệ thống loại này bao phủ nhiều ứng dụng, gồm bộ nhớ dịch (*translation memory*), hội thoại, tóm tắt, phân loại tài liệu, v.v.

Một lựa chọn thiết kế khác cho hệ thống bộ nhớ là xét, một phần hoặc toàn bộ, dạng liên tục cho các thành phần trên. Một phương pháp là mã hóa mỗi mục đầu vào thành một vectơ số thực (ví dụ một vectơ nhúng từ) nhưng vẫn dùng các mô-đun cập nhật và truy xuất như trong các hệ thống kiểu truy xuất thông tin thông thường [Weston và cộng sự, 2015; Khandelwal và cộng sự, 2019]. Một phương pháp khác là dùng các hàm khả vi cho mọi bước xây dựng và truy cập bộ nhớ. Các mô hình này thường được hiện thực bằng mạng nơ-ron và huấn luyện bằng hạ gradient [Sukhbaatar và cộng sự, 2015; Graves và cộng sự, 2014; Kumar và cộng sự, 2016; Graves và cộng sự, 2016; Miller và cộng sự, 2016]. Ý tưởng này thúc đẩy các nghiên cứu về cách kết hợp mạng nơ-ron với bộ nhớ, chẳng hạn **mạng bộ nhớ đầu-cuối** (*end-to-end memory networks*) và **máy**

Turing nơ-ron (*neural Turing machines*). Lưu ý rằng các mô hình trên đôi khi được gọi là **bộ nhớ ngoài** (*external memories*), vì chúng được dùng như các mô-đun riêng làm việc cùng các hệ thống khác.

Bộ nhớ cũng có thể đóng vai trò một thành phần bên trong hay ẩn của hệ thống. Trong trường hợp này, bộ nhớ thường được xây dựng lại cho mỗi mẫu đầu vào, nên có thể xem nó là một trường hợp của trí nhớ ngắn hạn. Có nhiều cách dùng loại bộ nhớ này để cải thiện các mô hình chuỗi. Trong phần còn lại của mục này, chúng ta tập trung vào việc dùng cơ chế bộ nhớ trong RNN. Ở Chương 5, ta sẽ thấy ý tưởng bộ nhớ được mở rộng thế nào để mô hình hóa sự tương ứng giữa các token của hai chuỗi.

Chú giải của người dịch: Bộ nhớ trong máy tính và trong mạng nơ-ron

Một thư viện là ví dụ dễ hình dung của bộ nhớ ngoài: *mã hóa* là viết thẻ mô tả cho mỗi cuốn sách, *cập nhật* là xếp sách lên đúng giá theo chỉ mục, *truy xuất* là tra thẻ để tìm sách khớp với yêu cầu. Còn trạng thái ẩn của RNN giống một trang nháp duy nhất: mọi thông tin bị "viết đè" lên cùng một chỗ, không có ngăn kéo hay chỉ mục, và được xóa sạch khi bắt đầu câu mới. Chữ "khả vi" nghĩa là tính được đạo hàm — điều kiện để có thể huấn luyện bằng hạ gradient.

4.3.2 Bộ nhớ dài-ngắn hạn

Trong RNN cơ bản trình bày ở Mục 4.2.1, phần tóm tắt các từ ngữ cảnh được cho bởi đầu ra của một đơn vị hồi quy. Nó ngầm định nghĩa một bộ nhớ, và do đó về nguyên tắc cho phép mô hình hóa các phụ thuộc có độ dài bất kỳ. Bộ nhớ này chỉ đơn giản kết hợp biểu diễn của lịch sử trước đó $w_1 \cdots w_{i-1}$ với đầu vào ở bước hiện tại w_i, mà không xét xem nên nén bao nhiêu thông tin từ các bước khác nhau vào một biểu diễn có độ dài cố định. Một vấn đề của mô hình này là khi dự đoán cần đến các phụ thuộc xa, bộ nhớ có thể không giữ đủ thông tin liên quan, và có thể khó học được các phụ thuộc này qua lan truyền ngược [Bengio và cộng sự, 1994; Pascanu và cộng sự, 2013]. Vì vậy, một cách tiếp cận mạnh hơn là tính toán xem nên giữ lại gì ở mỗi bước, và để mô hình tự học cách quyết định ghi nhớ hay quên đi.

Bộ nhớ dài-ngắn hạn (*long short-term memory, LSTM*) có lẽ là biến

thể nổi tiếng nhất của RNN để đạt được mục tiêu trên [Hochreiter và Schmidhuber, 1997]. Ý tưởng cơ bản của LSTM là một đơn vị hồi quy có thể học cách ghi nhớ những điều hữu ích và quên những điều vô ích bằng cách duy trì một bộ nhớ tường minh [Gers và cộng sự, 2000]. Để làm vậy, đơn vị hồi quy cơ bản được thay bằng một đơn vị LSTM gồm một vectơ đầu ra (gọi là **ô hồi quy** (*recurrent cell*)), một vectơ bộ nhớ (gọi là **ô nhớ** (*memory cell*)), và ba cổng để điều khiển dòng thông tin bên trong đơn vị LSTM. Là một mở rộng của RNN, mạng LSTM xử lý chuỗi đầu vào như thường lệ: nó bắt đầu với một số trạng thái khởi tạo, rồi lặp lại việc nhận một đầu vào và cho ra một vectơ. Một khác biệt then chốt giữa mạng LSTM và RNN là đơn vị LSTM ở bước i nhận cả ô hồi quy lẫn ô nhớ của bước trước nó. Dạng của một đơn vị LSTM là:

$$(\mathbf{h}_i, \mathbf{c}_i) \;=\; \text{LSTM}(\mathbf{h}_{i-1}, \mathbf{c}_{i-1}, \mathbf{x}_i), \tag{4.19}$$

trong đó $\mathbf{h}_i \in \mathbb{R}^{d_h}$ là ô hồi quy ở bước i, $\mathbf{c}_i \in \mathbb{R}^{d_h}$ là ô nhớ ở bước i, và $\mathbf{x}_i \in \mathbb{R}^{d_e}$ là đầu vào ở bước i. Khi đã có LSTM$(\cdot)$, việc áp dụng mô hình LSTM rất đơn giản. Ta chỉ cần gọi lặp lại LSTM$(\cdot)$ cho các đầu vào $\{\mathbf{x}_1, \ldots, \mathbf{x}_m\}$ và thu được các đầu ra $\{\mathbf{h}_1, \ldots, \mathbf{h}_m\}$. Điều này giống cách ta dùng RNN cơ bản, nên rất dễ mở rộng LSTM thành mô hình nhiều tầng (xem Mục 4.2.3) và mô hình hai chiều (xem Mục 4.2.4).

Ta có thể chia LSTM$(\cdot)$ thành ba bước.

- **Bước 1: Quên** (*Forget*). Giả sử $\mathbf{c}_{i-1}$ chứa thông tin mà mô hình ghi nhớ ở bước $i-1$, ta cần xác định bao nhiêu thông tin trong $\mathbf{c}_{i-1}$ bị loại bỏ khi xây dựng $\mathbf{c}_i$. Để làm điều này, một cổng được dùng để điều khiển mức độ quên ở từng chiều của $\mathbf{c}_{i-1}$. **Cổng quên** (*forget gate*) được định nghĩa là:

  $$\mathbf{f}_i \;=\; \text{Sigmoid}(\mathbf{h}_{i-1}\mathbf{U}_\text{f} + \mathbf{x}_i\mathbf{V}_\text{f} + \mathbf{b}_\text{f}), \tag{4.20}$$

  trong đó $\mathbf{f}_i \in [0,1]^{d_h}$ là một vectơ có cùng số chiều với $\mathbf{c}_{i-1}$. Hàm Sigmoid ánh xạ dữ liệu đầu vào vào khoảng $[0,1]$. Do đó, một phần tử của $\mathbf{f}_i$ cho biết phần tử tương ứng của $\mathbf{c}_{i-1}$ được giữ lại bao nhiêu. Xa hơn nữa, $\mathbf{f}_i \odot \mathbf{c}_{i-1}$ biểu diễn phần bộ nhớ được giữ lại sau khi đi qua cổng quên. Xem Hình 4.5(a) để thấy minh họa cổng quên trong đơn vị LSTM.

- **Bước 2: Cập nhật** (*Update*). Tiếp theo ta cập nhật bộ nhớ bằng cách xét cả trạng thái trước của bộ nhớ (tức $\mathbf{c}_{i-1}$) lẫn đầu vào của

đơn vị LSTM (tức $\mathbf{x}_i$ và $\mathbf{h}_{i-1}$). Trước tiên ta kết hợp $\mathbf{x}_i$ và $\mathbf{h}_{i-1}$ bằng một mạng nơ-ron đơn giản:

$$\hat{\mathbf{c}}_i = \text{TanH}(\mathbf{h}_{i-1}\mathbf{U}_\text{c} + \mathbf{x}_i\mathbf{V}_\text{c} + \mathbf{b}_\text{c}). \tag{4.21}$$

$\hat{\mathbf{c}}_i$ có thể được xem là thông tin mới mà ta định thêm vào bộ nhớ ở bước i. Một lần nữa, ta cần một cách điều khiển lượng thông tin đi vào bộ nhớ. Vì vậy ta định nghĩa một **cổng vào** (*input gate*) là

$$\mathbf{g}_i = \text{Sigmoid}(\mathbf{h}_{i-1}\mathbf{U}_\text{g} + \mathbf{x}_i\mathbf{V}_\text{g} + \mathbf{b}_\text{g}). \tag{4.22}$$

Phương trình này tương tự Phương trình (4.20) nhưng với các tham số khác. Sau đó ta định nghĩa $\mathbf{g}_i \odot \hat{\mathbf{c}}_i$ là thông tin mới thực sự mà ta quan tâm. Lấy cả $\mathbf{f}_i \odot \mathbf{c}_{i-1}$ và $\mathbf{g}_i \odot \hat{\mathbf{c}}_i$, ô nhớ ở bước i được cho bởi:

$$\mathbf{c}_i = \mathbf{f}_i \odot \mathbf{c}_{i-1} + \mathbf{g}_i \odot \hat{\mathbf{c}}_i. \tag{4.23}$$

Nói cách khác, ta quên đi một phần cái cũ trong $\mathbf{c}_{i-1}$ và ghi nhớ một phần cái mới trong $\hat{\mathbf{c}}_i$. Xem Hình 4.5(b) để thấy minh họa bước cập nhật.

- **Bước 3: Đầu ra** (*Output*). Ở bước cuối, ta sinh đầu ra $\mathbf{h}_i$ dựa trên bộ nhớ $\mathbf{c}_i$. Thay vì sao chép $\mathbf{c}_i$ sang $\mathbf{h}_i$, ta đưa $\mathbf{c}_i$ qua hàm tang hyperbolic rồi nhân kết quả với cổng ra. Giống các Phương trình (4.20) và (4.22), cổng ra được cho bởi:

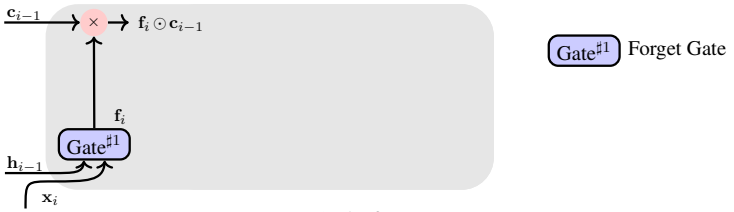
$$\mathbf{o}_i = \text{Sigmoid}(\mathbf{h}_{i-1}\mathbf{U}_\text{o} + \mathbf{x}_i\mathbf{V}_\text{o} + \mathbf{b}_\text{o}). \tag{4.24}$$

Khi đó, đầu ra của đơn vị LSTM được định nghĩa là:

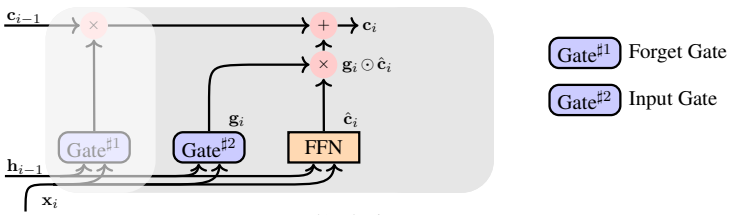
$$\mathbf{h}_i = \mathbf{o}_i \odot \text{TanH}(\mathbf{c}_i). \tag{4.25}$$

Xem Hình 4.5(c) để thấy minh họa bước đầu ra.

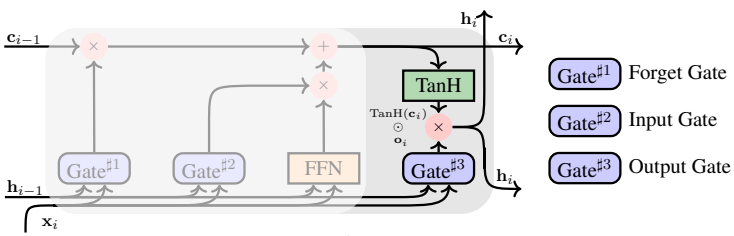
Mô hình LSTM được tham số hóa bởi $\mathbf{U}_\text{f}, \mathbf{U}_\text{c}, \mathbf{U}_\text{g}, \mathbf{U}_\text{o} \in \mathbb{R}^{d_h \times d_h}$, $\mathbf{V}_\text{f}, \mathbf{V}_\text{c}, \mathbf{V}_\text{g}, \mathbf{V}_\text{o} \in \mathbb{R}^{d_e \times d_h}$, và $\mathbf{b}_\text{f}, \mathbf{b}_\text{c}, \mathbf{b}_\text{g}, \mathbf{b}_\text{o} \in \mathbb{R}^{d_h}$. So với RNN cơ bản, các tham số bổ sung phát sinh trực tiếp từ kiến trúc dựa trên cổng này. Trong thực tế, có thể hiện thực đơn vị LSTM theo nhiều cách khác nhau, ví dụ dùng các hàm kích hoạt khác $\text{Sigmoid}(\cdot)$ và $\text{TanH}(\cdot)$, hay bỏ các số hạng độ lệch $\mathbf{b}_\text{f}$, $\mathbf{b}_\text{c}$, $\mathbf{b}_\text{g}$ và $\mathbf{b}_\text{o}$. Việc huấn luyện mô hình LSTM tuân theo khuôn mẫu chuẩn của việc huấn luyện các mô hình dựa trên RNN. Ví dụ, ta có thể xây dựng một mô hình ngôn ngữ dựa trên LSTM và huấn luyện nó bằng các phương pháp trình bày ở Mục 4.2.2.



(a) The forget gate.



(b) The input gate.



(c) The output gate.

Hình 4.5: Kiến trúc của đơn vị LSTM. Tại bước i, nó nhận đầu vào $\mathbf{x}_i$, rồi cập nhật cả ô nhớ ($\mathbf{c}_{i-1} \rightarrow \mathbf{c}_i$) lẫn ô hồi quy ($\mathbf{h}_{i-1} \rightarrow \mathbf{h}_i$). Quá trình này gồm ba cổng: cổng quên điều khiển bao nhiêu thông tin trong $\mathbf{c}_{i-1}$ được giữ lại ở bước i, cổng vào điều khiển bao nhiêu thông tin trong $\mathbf{c}_{i-1}$ và $\mathbf{x}_i$ được giữ lại ở bước i, và cổng ra điều khiển bao nhiêu thông tin trong $\mathbf{c}_i$ được dùng để tạo thành $\mathbf{h}_i$. (Nhãn trong hình: *Forget Gate* = cổng quên; *Input Gate* = cổng vào; *Output Gate* = cổng ra; (a) cổng quên; (b) cổng vào; (c) cổng ra.)

Chú giải của người dịch: Các "cổng" của LSTM hoạt động thế nào?

Ký hiệu $\odot$ là phép nhân *theo từng phần tử*: $[a_1\ a_2] \odot [b_1\ b_2] = [a_1 b_1\ \ a_2 b_2]$. Vì Sigmoid cho giá trị trong $[0, 1]$, mỗi phần tử của một cổng giống một "van nước": 0 là đóng hẳn, 1 là mở hết. Ví dụ với bộ nhớ 2 chiều: $\mathbf{c}_{i-1} = [5\ \ -2]$, cổng quên $\mathbf{f}_i = [1\ \ 0]$, thông tin mới $\hat{\mathbf{c}}_i = [0.5\ \ 0.8]$, cổng vào $\mathbf{g}_i = [0\ \ 1]$. Theo Phương trình (4.23): $\mathbf{c}_i = [1 \cdot 5\ \ 0 \cdot (-2)] + [0 \cdot 0.5\ \ 1 \cdot 0.8] = [5\ \ 0.8]$. Chiều thứ nhất giữ nguyên ký ức cũ, chiều thứ hai xóa ký ức cũ và nạp thông tin mới. Điểm mấu chốt: $\mathbf{c}_i$ được tính chủ yếu bằng *phép cộng* (thay vì đưa qua hàm kích hoạt mỗi bước như RNN cơ bản), nên khi cổng quên gần 1, thông tin — và gradient — có thể truyền qua nhiều bước mà không bị teo nhỏ. Đó là lý do LSTM giảm được vấn đề tiêu biến gradient.

4.3.3 Đơn vị hồi quy có cổng

Ở trên, ta đã thấy vai trò quan trọng của các đơn vị cổng và ô nhớ. Nhìn chung, việc dùng các mạng nơ-ron này khiến mô hình tốn kém hơn về tính toán. Một lựa chọn thay thế hiệu quả hơn về tính toán cho LSTM là **đơn vị hồi quy có cổng** (*gated recurrent unit, GRU*), dùng một cấu trúc mô hình đơn giản hơn với ít hàm cổng hơn [Cho và cộng sự, 2014; Chung và cộng sự, 2014]. Khác với LSTM, GRU không có ô nhớ, nên giống như một đơn vị RNN, nó nhận vectơ trạng thái trước $\mathbf{h}_{i-1}$ và vectơ đầu vào hiện tại $\mathbf{x}_i$, rồi sinh ra vectơ trạng thái hiện tại $\mathbf{h}_i$.

Kiến trúc GRU gồm hai đơn vị cổng: **cổng đặt lại** (*reset gate*) và **cổng cập nhật** (*update gate*). Cổng đặt lại, đúng như tên gọi, được dùng để đặt lại (hoặc co giãn lại) trạng thái của GRU (tức $\mathbf{h}_{i-1}$). Tương tự các cổng trong LSTM, cổng đặt lại được định nghĩa là:

$$\mathbf{r}_i \ = \ \text{Sigmoid}(\mathbf{h}_{i-1}\mathbf{U}_\text{r} + \mathbf{x}_i\mathbf{V}_\text{r} + \mathbf{b}_\text{r}), \tag{4.26}$$

trong đó $\mathbf{r}_i \in [0, 1]^{d_h}$ là một vectơ các số vô hướng, mỗi chiều mô tả bao nhiêu thông tin ở chiều tương ứng của $\mathbf{h}_{i-1}$ được giữ lại. Như vậy, ta có một biểu diễn của thông tin được giữ lại

$$\upsilon_{i-1} \ = \ \mathbf{r}_i \odot \mathbf{h}_{i-1}. \tag{4.27}$$

Lấy cả thông tin được giữ lại υ_{i-1} và đầu vào hiện tại $\mathbf{x}_i$, một vectơ trạng

thái mới được định nghĩa là:

$$\hat{\mathbf{h}}_i \;=\; \mathrm{TanH}(\upsilon_{i-1}\mathbf{U}_\mathrm{h} + \mathbf{x}_i\mathbf{V}_\mathrm{h} + \mathbf{b}_\mathrm{h}). \tag{4.28}$$

Cổng cập nhật khi đó được cho bởi:

$$\mathbf{u}_i \;=\; \mathrm{Sigmoid}(\mathbf{h}_{i-1}\mathbf{U}_\mathrm{u} + \mathbf{x}_i\mathbf{V}_\mathrm{u} + \mathbf{b}_\mathrm{u}), \tag{4.29}$$

$\mathbf{u}_i$ có thể được xem là một vectơ trọng số nội suy, cân bằng giữa việc giữ lại trạng thái cũ và việc tích hợp trạng thái ứng viên mới. Cuối cùng, đầu ra của GRU được định nghĩa là một phép nội suy tuyến tính giữa $\hat{\mathbf{h}}_i$ và $\mathbf{h}_{i-1}$

$$\mathbf{h}_i \;=\; \mathbf{u}_i \odot \hat{\mathbf{h}}_i + (1 - \mathbf{u}_i) \odot \mathbf{h}_{i-1}. \tag{4.30}$$

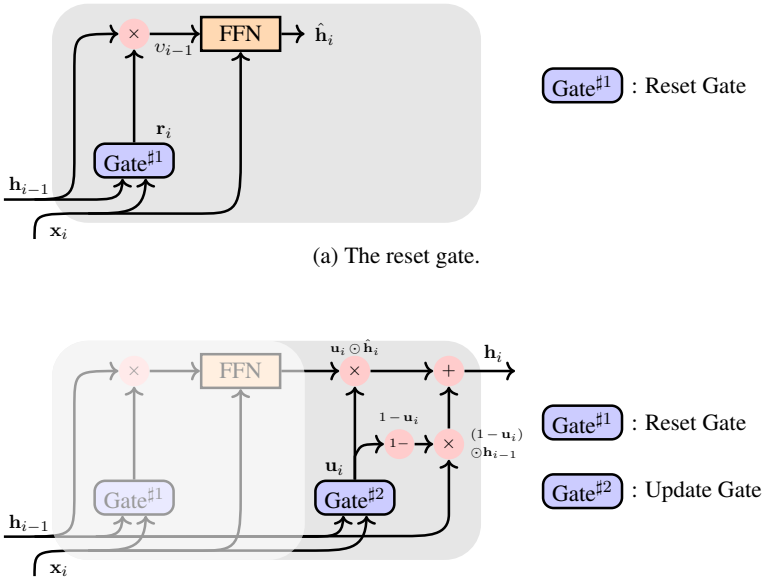
Hình 4.6 minh họa dòng thông tin trong một đơn vị GRU. Các tham số ở đây là $\mathbf{U}_\mathrm{r}, \mathbf{U}_\mathrm{h}, \mathbf{U}_\mathrm{u} \in \mathbb{R}^{d_h \times d_h}$, $\mathbf{V}_\mathrm{r}, \mathbf{V}_\mathrm{h}, \mathbf{V}_\mathrm{u} \in \mathbb{R}^{d_e \times d_h}$, và $\mathbf{b}_\mathrm{r}, \mathbf{b}_\mathrm{h}, \mathbf{b}_\mathrm{u} \in \mathbb{R}^{d_h}$. Do đó, mô hình GRU nhỏ hơn mô hình LSTM vì dùng ít đơn vị cổng hơn. Lưu ý rằng việc bỏ ô nhớ khiến GRU hiệu quả hơn. Trong trường hợp này, vai trò của bộ nhớ được đảm nhận một cách ngầm định bởi trạng thái ẩn $\mathbf{h}_i$, vốn được cập nhật linh động để lưu giữ các thông tin quan trọng theo thời gian.

Chú giải của người dịch: Nội suy tuyến tính trong GRU

Phương trình (4.30) giống công thức "trung bình có trọng số" quen thuộc: với hai số a, b và $t \in [0, 1]$, $t \cdot a + (1 - t) \cdot b$ là một điểm nằm giữa a và b ($t = 1$ cho a, $t = 0$ cho b). Ví dụ một chiều: $\hat{h}_i = 0.8$, $h_{i-1} = -0.4$, $u_i = 0.25$ thì $h_i = 0.25 \cdot 0.8 + 0.75 \cdot (-0.4) = -0.1$, tức trạng thái mới vẫn nghiêng nhiều về trạng thái cũ. GRU áp dụng điều này cho từng chiều với trọng số riêng. So về số ma trận: LSTM có 4 bộ ($\mathbf{U}, \mathbf{V}, \mathbf{b}$), còn GRU chỉ có 3, nên GRU nhỏ hơn khoảng một phần tư.

4.4 Các mô hình tích chập

Trong mục này, chúng ta mô tả một loại mô hình khác dùng cho mô hình hóa chuỗi, gọi là mạng nơ-ron tích chập (*convolutional neural networks, CNNs*). Phần tổng quan dưới đây trình bày các khái niệm cơ bản, nhưng không nhằm giới thiệu đầy đủ mọi biến thể CNN và các kỹ thuật tiên tiến nhất. Thay vào đó, chúng ta tập trung vào việc áp dụng CNN cho dữ liệu dạng chuỗi và trình bày một số cải tiến về kiến trúc.



(a) The reset gate.

(b) The update gate.

Hình 4.6: Kiến trúc của GRU. Khác với đơn vị LSTM, GRU không có ô nhớ, do đó tuân theo cùng dạng đầu vào và đầu ra như một đơn vị RNN tiêu chuẩn. GRU có hai cổng. Cổng đặt lại điều khiển bao nhiêu thông tin trong $\mathbf{h}_{i-1}$ được giữ lại ở bước i. Thông tin được giữ lại sau đó được hợp nhất với đầu vào $\mathbf{x}_i$, sinh ra đầu ra ứng viên $\hat{\mathbf{h}}_i$. Cổng cập nhật tìm sự cân bằng giữa $\hat{\mathbf{h}}_i$ và $\mathbf{h}_{i-1}$ khi tính đầu ra cuối cùng của GRU. (Nhãn trong hình: *Reset Gate* = cổng đặt lại; *Update Gate* = cổng cập nhật; (a) cổng đặt lại; (b) cổng cập nhật.)

4.4.1 Tích chập

Một tính chất then chốt của CNN là *kiến trúc chia sẻ trọng số*: một nhân (*kernel*) hay bộ lọc (*filter*) trượt trên dữ liệu đầu vào và tạo ra một bản đồ đặc trưng (*feature map*). Ý tưởng là tại mỗi thời điểm, bộ lọc chỉ nhận tín hiệu từ một vùng giới hạn của dữ liệu (gọi là trường tiếp nhận (*receptive field*)), và tính tổng có trọng số của các tín hiệu đầu vào này. Để minh họa, ta xét trường hợp thường gặp là bộ lọc trong CNN được dùng để xử lý dữ liệu hai chiều. Xét ma trận dữ liệu 3×3:

$$\mathbf{A} = \begin{bmatrix} 1 & 9 & 7 \\ 3 & 1 & 2 \\ 0 & 1 & -1 \end{bmatrix}, \tag{4.31}$$

và một bộ lọc 2×2 có ma trận trọng số

$$\mathbf{W} = \begin{bmatrix} 2 & 0 \\ 2 & 2 \end{bmatrix}. \tag{4.32}$$

Ta áp dụng bộ lọc này lên từng ma trận con 2×2 của $\mathbf{A}$ (trong ví dụ này có bốn ma trận con như vậy) và tính tổng các phần tử có trọng số theo $\mathbf{W}$. Chẳng hạn, với ma trận con 2×2 ở góc trên bên trái của $\mathbf{A}$, đầu ra của bộ lọc được tính như sau:

$$\begin{aligned} \mathrm{Conv}\Big(\begin{bmatrix} 1 & 9 \\ 3 & 1 \end{bmatrix}, \mathbf{W} \Big) &= \mathrm{Conv}\Big(\begin{bmatrix} 1 & 9 \\ 3 & 1 \end{bmatrix}, \begin{bmatrix} 2 & 0 \\ 2 & 2 \end{bmatrix} \Big) \\ &= 1 \times 2 + 9 \times 0 + 3 \times 2 + 1 \times 2 \\ &= 10. \end{aligned} \tag{4.33}$$

$\mathrm{Conv}(\cdot)$ định nghĩa một **phép tích chập** (*convolution operation*), tức là lấy tổng các phần tử của tích theo từng phần tử của hai ma trận. Phép tích chập có thể được mở rộng để phủ toàn bộ ma trận đầu vào bằng cách trượt bộ lọc trên nó, như sau:

$$\mathrm{Conv}(\mathbf{A}, \mathbf{W}) = \mathrm{Conv}\Big(\begin{bmatrix} 1 & 9 & 7 \\ 3 & 1 & 2 \\ 0 & 1 & -1 \end{bmatrix}, \mathbf{W} \Big)$$

$$= \begin{bmatrix} \text{Conv}\Big(\begin{bmatrix} 1 & 9 \\ 3 & 1 \end{bmatrix}, \mathbf{W}\Big) & \text{Conv}\Big(\begin{bmatrix} 9 & 7 \\ 1 & 2 \end{bmatrix}, \mathbf{W}\Big) \\ \text{Conv}\Big(\begin{bmatrix} 3 & 1 \\ 0 & 1 \end{bmatrix}, \mathbf{W}\Big) & \text{Conv}\Big(\begin{bmatrix} 1 & 2 \\ 1 & -1 \end{bmatrix}, \mathbf{W}\Big) \end{bmatrix}$$

$$= \begin{bmatrix} 10 & 24 \\ 8 & 2 \end{bmatrix}. \tag{4.34}$$

Đầu ra $\begin{bmatrix} 10 & 24 \\ 8 & 2 \end{bmatrix}$ còn được gọi là **bản đồ đặc trưng** (*feature map*) của bộ lọc $\mathbf{W}$ trên $\mathbf{A}$. Đôi khi, phép tích chập $\text{Conv}(\mathbf{A}, \mathbf{W})$ được viết là $\mathbf{A} * \mathbf{W}$, trong đó ký hiệu $*$ biểu thị **tích chập** (*convolution product*)[8].

Chú giải của người dịch: Tích chập = "trượt và nhân–cộng"

Hãy hình dung bộ lọc $\mathbf{W}$ như một "tấm kính lúp" cỡ 2×2 đặt lên ma trận $\mathbf{A}$. Ở mỗi vị trí, ta nhân từng ô của kính với ô nằm ngay bên dưới rồi cộng tất cả lại để được *một số*. Sau đó dịch kính sang phải (hoặc xuống dưới) một ô và làm lại. Kiểm tra ô góc trên bên phải của Phương trình (4.34): ma trận con là $\begin{bmatrix} 9 & 7 \\ 1 & 2 \end{bmatrix}$, nên $9 \times 2 + 7 \times 0 + 1 \times 2 + 2 \times 2 = 18 + 0 + 2 + 4 = 24$. Vì bộ lọc 2×2 chỉ đặt được ở $2 \times 2 = 4$ vị trí trên ma trận 3×3, bản đồ đặc trưng có kích thước 2×2. Điểm quan trọng: *cùng một* bộ trọng số $\mathbf{W}$ được dùng ở mọi vị trí — đó là ý nghĩa của "chia sẻ trọng số".

Bây giờ ta xét một mô tả tổng quát hơn về tích chập trong CNN. Giả

[8] Trong giải tích toán học, cho hai hàm khả tích $f(\cdot)$ và $g(\cdot)$, **tích chập** (*convolution*) định nghĩa một hàm khả tích mới $f * g(\cdot)$ mô tả tích phân của $f(\cdot)$ được đánh trọng số bởi hàm $g(\cdot)$ đã được lấy đối xứng và tịnh tiến. Chính xác hơn, tích chập của các hàm liên tục được định nghĩa là:

$$f * g(x) = \int_{\mathbb{R}} f(y)\, g(x - y)\, \mathrm{d}y, \tag{4.35}$$

trong đó $f(y)$ là hàm mà ta quan tâm, còn $g(x - y)$ là hàm trọng số, thu được bằng cách lấy đối xứng $g(y)$ qua trục y rồi tịnh tiến nó một đoạn x. Một trường hợp đặc biệt là khi x và y đều là số nguyên. Khi đó, ta có thể định nghĩa $f * g(\cdot)$ là

$$f * g(x) = \sum_{y} f(y)\, g(x - y), \tag{4.36}$$

đây chính là dạng cơ bản của Phương trình (4.33). Trong CNN, x, y và $x - y$ có thể được xem là chỉ số của các phần tử trong mảng dữ liệu. $f(y)$ là một phần tử dữ liệu trong mảng đầu vào, và $g(x - y)$ là trọng số tương ứng trong bộ lọc. Dùng Phương trình (4.36), ta tính được giá trị của phần tử có chỉ số x trong mảng đầu ra $f * g(x)$ (tức là bản đồ đặc trưng).

sử $\mathbf{A}$ là một mảng dữ liệu nhiều chiều. Một bộ lọc định nghĩa một cửa sổ (hay trường tiếp nhận) trên $\mathbf{A}$. Ta có thể di chuyển cửa sổ trên $\mathbf{A}$ theo các hướng khác nhau. Việc này tạo ra một tập các mảng dữ liệu, ký hiệu là Ω. Mỗi mảng dữ liệu $\mathbf{a}_p \in \Omega$ được tạo thành từ các phần tử thuộc vùng tương ứng của $\mathbf{A}$. Ví dụ, có bốn ma trận con trong Phương trình (4.34): $\mathbf{a}_1 = \begin{bmatrix} 1 & 9 \\ 3 & 1 \end{bmatrix}$, $\mathbf{a}_2 = \begin{bmatrix} 9 & 7 \\ 1 & 2 \end{bmatrix}$, $\mathbf{a}_3 = \begin{bmatrix} 3 & 1 \\ 0 & 1 \end{bmatrix}$ và $\mathbf{a}_4 = \begin{bmatrix} 1 & 2 \\ 1 & -1 \end{bmatrix}$. Ngoài ra, ta giả sử bộ lọc được tham số hóa bởi một mảng trọng số $\mathbf{W}$ có cùng kích thước với $\mathbf{a}_p$, tức là $|\mathbf{a}_p| = |\mathbf{W}|$. Kết quả của việc áp dụng bộ lọc lên $\mathbf{A}$ là một mảng các đặc trưng

$$\text{Conv}(\mathbf{A}, \mathbf{W}) = \begin{bmatrix} v_1 & \cdots & v_{|\Omega|} \end{bmatrix}. \tag{4.37}$$

Mỗi đặc trưng v_p được cho bởi:

$$\begin{aligned} v_p &= \text{Conv}(\mathbf{a}_p, \mathbf{W}) \\ &= \mathbf{a}_p \cdot \mathbf{W} \\ &= \sum_{k=1}^{|\mathbf{W}|} a_p(k) \cdot W(k), \end{aligned} \tag{4.38}$$

trong đó $a_p(k)$ và $W(k)$ lần lượt là phần tử thứ k của các mảng đã được duỗi phẳng $\mathbf{a}_p$ và $\mathbf{W}$. Lưu ý rằng mặc dù mảng $\begin{bmatrix} v_1 & \dots & v_{|\Omega|} \end{bmatrix}$ có thể được định dạng lại thành nhiều dạng khác nhau (như ma trận hay tensor 3 chiều) tùy theo số chiều của đầu vào, các biểu diễn này về cơ bản là tương đương xét theo góc độ lưu trữ dữ liệu. Chẳng hạn, áp dụng một bộ lọc hai chiều lên dữ liệu đầu vào hai chiều sẽ cho một ma trận đặc trưng hai chiều, như minh họa trong Phương trình (4.34).

Chú giải của người dịch: "Duỗi phẳng" một mảng

Duỗi phẳng (*flatten*) nghĩa là đọc các phần tử của một ma trận lần lượt theo từng hàng và xếp thành một dãy số. Ví dụ $\mathbf{a}_1 = \begin{bmatrix} 1 & 9 \\ 3 & 1 \end{bmatrix}$ thành $(1, 9, 3, 1)$ và $\mathbf{W}$ thành $(2, 0, 2, 2)$. Khi đó Phương trình (4.38) chính là tích vô hướng quen thuộc của hai vectơ 4 chiều: $1 \cdot 2 + 9 \cdot 0 + 3 \cdot 2 + 1 \cdot 2 = 10$, giống như tích vô hướng $\vec{u} \cdot \vec{v} = x_1 x_2 + y_1 y_2$ ở lớp 10 nhưng có nhiều tọa độ hơn.

Hơn nữa, ta cần giải quyết hai vấn đề để mô hình dùng được trong thực tế. Thứ nhất, ta phải xác định **bước trượt** (*stride*), tức là độ dài mỗi

bước dịch chuyển của bộ lọc khi nó trượt trên $\mathbf{A}$. Trong ví dụ trước, ta đã giả định bước trượt bằng 1 (tức là stride $= 1$). Bằng cách dùng bước trượt lớn hơn, ta thực chất đã giảm mẫu (*downsample*) $\mathbf{A}$, nén nó thành một số lượng đặc trưng ít hơn. Thứ hai, để kiểm soát kích thước không gian của bản đồ đặc trưng đầu ra, ta có thể thêm các phần tử giả (hay **phần đệm** (*padding*)) quanh biên của dữ liệu đầu vào. Đệm số 0 (*zero-padding*), tức là bao quanh vùng đầu vào bằng các số 0, là cách làm phổ biến nhất. Ví dụ, xét ma trận dữ liệu 2×2:

$$\mathbf{A} = \begin{bmatrix} 1 & 9 \\ 7 & 3 \end{bmatrix}. \tag{4.39}$$

Ta có thể thêm các phần tử có giá trị 0 xung quanh nó để được một ma trận 4×4

$$\mathbf{A}_{\text{padding}} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 9 & 0 \\ 0 & 7 & 3 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}. \tag{4.40}$$

Dùng cùng bộ lọc như trong Phương trình (4.33) với stride $= 1$, ta có bản đồ đặc trưng 3×3

$$\text{Conv}_{\text{stride}=1}(\mathbf{A}_{\text{padding}}, \mathbf{W}) = \begin{bmatrix} 2 & 20 & 18 \\ 14 & 22 & 24 \\ 0 & 14 & 6 \end{bmatrix}. \tag{4.41}$$

Nếu stride $= 2$ thì ta sẽ có bản đồ đặc trưng có cùng kích thước với dữ liệu đầu vào

$$\text{Conv}_{\text{stride}=2}(\mathbf{A}_{\text{padding}}, \mathbf{W}) = \begin{bmatrix} 2 & 18 \\ 0 & 6 \end{bmatrix}. \tag{4.42}$$

Chú giải của người dịch: Kích thước đầu ra

Với đầu vào (đã đệm) có cạnh dài n, bộ lọc cạnh f và bước trượt s, số vị trí đặt được bộ lọc trên mỗi cạnh là $\lfloor (n-f)/s \rfloor + 1$. Ở Phương trình (4.41): $n = 4$, $f = 2$, $s = 1$ cho $(4-2)/1+1 = 3$, nên đầu ra là 3×3. Ở Phương trình (4.42): $s = 2$ cho $(4-2)/2+1 = 2$, nên đầu ra là 2×2 — bộ lọc "nhảy cóc" 2 ô mỗi lần, chỉ dừng ở các góc trên-trái, trên-phải, dưới-trái, dưới-phải.

Kiểm tra ô đầu tiên: ma trận con $\begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$ cho $0 \cdot 2 + 0 \cdot 0 + 0 \cdot 2 + 1 \cdot 2 = 2$.

Lưu ý rằng, nói một cách chặt chẽ, phép toán được cài đặt trong học sâu về mặt kỹ thuật là **tương quan chéo** (*cross-correlation*) chứ không phải tích chập thực sự theo nghĩa toán học. Tích chập thực sự đòi hỏi bộ lọc phải được lấy đối xứng (lật) trước khi trượt trên đầu vào. Tuy nhiên, vì trọng số của bộ lọc trong CNN được học từ đầu, bước lật này là thừa về mặt toán học. Hầu hết các framework học sâu bỏ qua bước này để tính toán hiệu quả hơn, dù trong lĩnh vực này người ta vẫn quen giữ thuật ngữ "tích chập".

4.4.2 CNN cho mô hình hóa chuỗi

Theo cách trình bày ở các mục trước, ta giả sử đầu vào của một mô hình chuỗi là một chuỗi vectơ $\mathbf{x}_1 \cdots \mathbf{x}_m$ và đầu ra là một chuỗi vectơ khác $\mathbf{h}_1 \cdots \mathbf{h}_m$. Ví dụ, ta có thể xem $\mathbf{x}_1 \cdots \mathbf{x}_m$ như một ma trận $\mathbf{X} \in \mathbb{R}^{m \times d_e}$, trong đó vectơ hàng thứ i biểu diễn từ thứ i của chuỗi.

Việc thực hiện tích chập trên $\mathbf{X}$ khá đơn giản. Vì $\mathbf{x}_i$ không áp đặt thứ tự nào giữa các chiều đặc trưng, ta không cần thực hiện tích chập dọc theo các chiều đặc trưng. Do đó, ta có thể dùng một trường tiếp nhận kích thước $r \times d_e$ và xét đồng thời tất cả các chiều của $\mathbf{x}_i$ trong phép tích chập. Trong ứng dụng thực tế, người ta thường dùng nhiều bộ lọc để nắm bắt các khía cạnh khác nhau của đầu vào. Chẳng hạn, có thể dùng một bộ lọc có trường tiếp nhận lớn để đưa vào ngữ cảnh rộng hơn, đồng thời dùng một bộ lọc khác có trường tiếp nhận nhỏ để tập trung vào các đặc trưng rất cục bộ. Hình 4.7 minh họa việc áp dụng hai bộ lọc khác nhau lên một chuỗi đầu vào.

Để phân phối các đặc trưng đã trích xuất vào $\{\mathbf{h}_1, \ldots, \mathbf{h}_m\}$, ta gắn mỗi lần áp dụng bộ lọc với một vị trí cụ thể trong chuỗi. Để bảo đảm chuỗi đầu vào và chuỗi đầu ra có cùng độ dài, một vectơ đệm được thêm vào cả hai đầu của chuỗi đầu vào. Bảng dưới đây cho thấy đầu vào và đầu ra của một CNN trên một chuỗi ví dụ:

Vị trí	Đầu vào	Trường tiếp nhận	Đầu ra
0	$\mathbf{x}_0 (= 0)$	Không có	Không có
1	$\mathbf{x}_1$	$\{\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2\}$	$\mathbf{h}_1$
2	$\mathbf{x}_2$	$\{\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3\}$	$\mathbf{h}_2$
3	$\mathbf{x}_3$	$\{\mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_4\}$	$\mathbf{h}_3$
4	$\mathbf{x}_4$	$\{\mathbf{x}_3, \mathbf{x}_4, \mathbf{x}_5\}$	$\mathbf{h}_4$
5	$\mathbf{x}_5 (= 0)$	Không có	Không có

Thông thường, một hàm kích hoạt được áp dụng để đưa tính phi tuyến vào đầu ra cuối cùng. Bằng cách này, ta xây dựng được một tầng tích chập (*convolutional layer*) tiêu chuẩn, có thể xem như một chuỗi các mạng nơ-ron kết nối đầy đủ, mỗi mạng nhận đầu vào từ một cửa sổ có kích thước cố định. Tại vị trí thứ i, đầu ra của tầng tích chập được cho bởi:

$$v_i = \psi(\text{Conv}(\mathbf{a}_i, \mathbf{W})), \tag{4.43}$$

trong đó $\mathbf{a}_i$ biểu diễn các đầu vào trong trường tiếp nhận[9], và $\mathbf{W}$ biểu diễn các tham số của bộ lọc. Trong trường hợp có nhiều bộ lọc (ví dụ, d_h bộ lọc), ta có một tập tham số $\{\mathbf{W}^{(1)}, \ldots, \mathbf{W}^{(d_h)}\}$, một tập hàm kích hoạt $\{\psi^{(1)}, \ldots, \psi^{(d_h)}\}$, và một tập đầu vào $\{\mathbf{a}_i^{(1)}, \ldots, \mathbf{a}_i^{(d_h)}\}$. Mỗi bộ ba $(\mathbf{W}^{(k)}, \psi^{(k)}, \mathbf{a}_i^{(k)})$ cho một đầu ra

$$v_i^{(k)} = \psi^{(k)}(\text{Conv}(\mathbf{a}_i^{(k)}, \mathbf{W}^{(k)})). \tag{4.44}$$

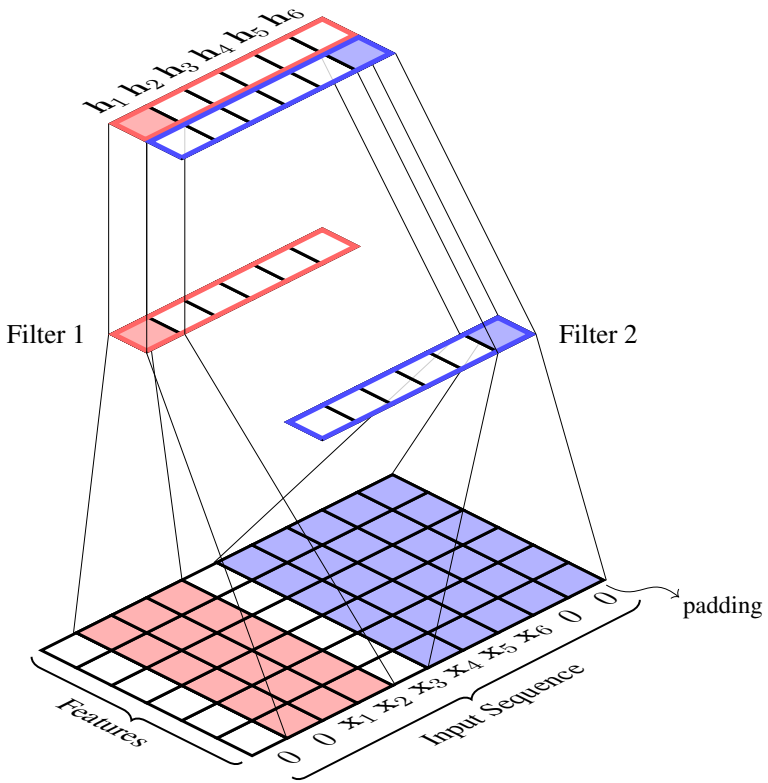
Lưu ý rằng $v_i^{(k)}$ chỉ đơn giản là một phần tử của $\mathbf{h}_i$. Do đó, $\mathbf{h}_i$ có thể được viết là

$$\mathbf{h}_i = \begin{bmatrix} v_i^{(1)} & \cdots & v_i^{(d_h)} \end{bmatrix}. \tag{4.45}$$

Chú giải của người dịch: Đọc Hình 4.7

Trong hình, *Features* là trục các chiều đặc trưng (6 chiều của mỗi vectơ từ), *Input Sequence* là trục vị trí trong chuỗi, *padding* là các từ đệm (toàn số 0). Mỗi bộ lọc "ôm" trọn cả 6 chiều đặc trưng và chỉ trượt dọc theo chuỗi. Mỗi bộ lọc cho ra *một số* tại mỗi vị trí; vì có 2 bộ lọc nên mỗi vị trí i nhận được 2 số, ghép lại thành vectơ $\mathbf{h}_i$ có $d_h = 2$ chiều (Phương trình (4.45)). Muốn

[9] Với trường tiếp nhận kích thước $r \times d_e$, $\mathbf{a}_i$ được định nghĩa là $\{\mathbf{x}_{\lceil i-\frac{r}{2} \rceil}, \ldots, \mathbf{x}_{\lceil i+\frac{r}{2}-1 \rceil}\}$ hoặc $\{\mathbf{x}_{\lfloor i-\frac{r}{2} \rfloor}, \ldots, \mathbf{x}_{\lfloor i+\frac{r}{2}-1 \rfloor}\}$.



Hình 4.7: Hai bộ lọc được áp dụng lên một chuỗi vectơ từ. Đầu vào gồm mười vectơ từ (các từ $\mathbf{x}_1 \cdots \mathbf{x}_6$ và hai từ đệm ở mỗi đầu của chuỗi). Mỗi vectơ từ có 6 chiều, do đó đầu vào là một ma trận 10×6. Bộ lọc 1 (*Filter 1*) có trường tiếp nhận kích thước 3×6. Trượt nó trên ma trận đầu vào, ta thu được một chuỗi đầu ra, mỗi đầu ra ứng với một vị trí (tức là một chuỗi 8 đầu ra). Tương tự, ta áp dụng bộ lọc 2 (*Filter 2*) lên chuỗi đầu vào và thu được một chuỗi đầu ra khác. Hai chuỗi đầu ra sau đó được tổ chức thành một ma trận 2×6, trong đó vectơ hàng thứ i là $\mathbf{h}_i$.

$\mathbf{h}_i$ có 512 chiều thì dùng 512 bộ lọc.

Nhiều hệ thống dựa trên CNN có ý nghĩa thực tiễn gồm hai tầng tích chập trở lên. Cách đơn giản nhất để làm điều này là xếp chồng các tầng, tương tự như RNN nhiều tầng (xem Mục 4.2.3). Cụ thể, chuỗi đầu ra của một tầng tích chập phía dưới được dùng trực tiếp làm đầu vào cho tầng tiếp theo. Hình 4.8 đưa ra một ví dụ về kiến trúc CNN gồm ba tầng tích chập xếp chồng. Một ưu điểm chính của CNN nhiều tầng là trường tiếp nhận của chúng được mở rộng theo cấp số nhân, dẫn đến phạm vi biểu diễn rộng hơn. Như minh họa trong Hình 4.8, trong khi một nơ-ron ở Tầng 1 chỉ tổng hợp thông tin từ ba vectơ đầu vào, một nơ-ron ở Tầng 3 tổng hợp gián tiếp thông tin trải trên bảy vectơ đầu vào. Vì các nơ-ron ở tầng cao hơn xử lý tín hiệu từ một đoạn liên tiếp dài hơn nhiều của chuỗi gốc, chúng có khả năng trích xuất các biểu diễn ngữ nghĩa ở mức cao hơn và nắm bắt hiệu quả các phụ thuộc khoảng cách xa (*long-distance dependencies*).

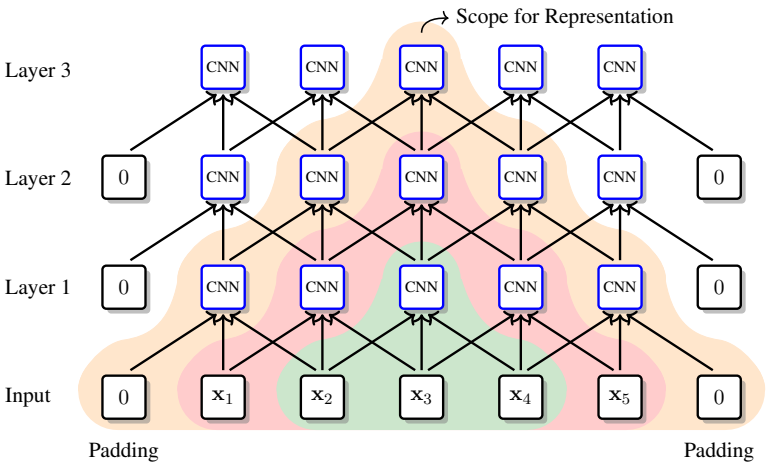
Chú giải của người dịch: Trường tiếp nhận lớn dần qua các tầng

Với bộ lọc rộng $r = 3$ và bước trượt 1, mỗi tầng mới "nhìn" thêm 1 phần tử ở mỗi bên. Tầng 1 thấy 3 phần tử, tầng 2 thấy $3 + 2 = 5$, tầng 3 thấy $5 + 2 = 7$; tổng quát, tầng L thấy $2L + 1$ phần tử đầu vào. Với các biến thể như tích chập giãn (*dilated convolution*), trong đó khoảng cách giữa các phần tử mà bộ lọc lấy vào được nhân đôi sau mỗi tầng, trường tiếp nhận còn có thể tăng gấp đôi qua mỗi tầng.

Trong một số ứng dụng, ta cần một biểu diễn có độ dài cố định và số chiều thấp cho toàn bộ chuỗi. Một cách phổ biến là thêm một tầng gộp (*pooling layer*) để hợp nhất $\{\mathbf{h}_1, \ldots, \mathbf{h}_m\}$ thành một vectơ duy nhất. Ví dụ, ta có thể chọn giá trị lớn nhất (tức là gộp cực đại (*max-pooling*)) hoặc lấy trung bình các giá trị (tức là gộp trung bình (*average-pooling*)) theo từng chiều. Đây là một phương pháp chung trong học máy và áp dụng được cho hầu hết các mô hình chuỗi được bàn đến trong cuốn sách này.

Chú giải của người dịch: Ví dụ gộp

Giả sử chuỗi có 3 vị trí với $\mathbf{h}_1 = (1,\ 5), \mathbf{h}_2 = (4,\ 2), \mathbf{h}_3 = (3,\ -1)$. Gộp cực đại lấy số lớn nhất theo từng chiều: $(\max(1, 4, 3),\ \max(5, 2, -1)) = (4,\ 5)$. Gộp trung bình: $((1 + 4 + 3)/3,\ (5 + 2 - 1)/3) = (8/3,\ 2)$. Dù chuỗi dài



Hình 4.8: Một CNN với 3 tầng tích chập (stride $= 1$ và $r = 3$). Mỗi tầng nhận một chuỗi vectơ và tạo ra một chuỗi vectơ khác. Trong quá trình này, một bộ lọc di chuyển trên đầu vào và thực hiện phép tích chập ở mỗi lần di chuyển. Ở tầng 1, trường tiếp nhận của bộ lọc là một vùng gồm ba phần tử đầu vào (xem vùng tô xanh lá). Tầng càng cao thì trường tiếp nhận của bộ lọc càng lớn. Ví dụ, ở tầng 3, một lần áp dụng bộ lọc có thể phủ tối đa toàn bộ chuỗi đầu vào (xem vùng tô màu cam). (*Layer*: tầng; *Input*: đầu vào; *Padding*: phần đệm; *Scope for Representation*: phạm vi biểu diễn.)

bao nhiêu, kết quả luôn là một vectơ có số chiều cố định (ở đây là 2).

4.4.3 Xử lý thông tin vị trí

Một tính chất hấp dẫn của CNN là khả năng cân bằng giữa độ phức tạp và hiệu quả bằng cách giới hạn kết nối đầy đủ trong một vùng cục bộ của dữ liệu đầu vào. Tính kết nối cục bộ này cho phép ta mô hình hóa một tầng tích chập về mặt toán học tương tự như một tầng kết nối đầy đủ: đầu ra của một nơ-ron là một biến đổi phi tuyến của tổng có trọng số các đầu vào của nó. Mặc dù kết nối cục bộ mang lại hiệu quả tính toán, một hạn chế của CNN tiêu chuẩn là tính bất biến đối với phép tịnh tiến (*translation invariance*). Dù nắm bắt được các cấu trúc cục bộ bên trong trường tiếp nhận, chúng không nhận biết được vị trí tuyệt đối mang tính toàn cục trong chuỗi.

Điều thú vị là, nếu chỉ giới hạn trong mô hình hóa chuỗi, điều này có vẻ không thành vấn đề vì đầu ra của mô hình bản thân nó cũng là một chuỗi. Có vẻ hợp lý khi giả định rằng chuỗi đầu ra giữ được thông tin thứ tự của chuỗi đầu vào. Tuy nhiên, việc áp dụng CNN cho dữ liệu chuỗi không bảo đảm một ánh xạ một-một chặt chẽ giữa các phần tử đầu vào và đầu ra. Về mặt kỹ thuật, $\mathbf{h}_i$ không đơn thuần là biểu diễn của $\mathbf{x}_i$. Thay vào đó, nó mã hóa một cửa sổ các đầu vào có tâm tại $\mathbf{x}_i$. Điều này làm vấn đề phức tạp lên đáng kể, vì rất khó suy ra từ $\mathbf{h}_i$ chính xác các đầu vào cục bộ đó ban đầu được sắp xếp theo thứ tự nào.

Chú giải của người dịch: Vì sao CNN "không biết" thứ tự?

Vì cùng một bộ lọc được dùng ở mọi vị trí, cụm "con mèo đen" cho ra cùng một đặc trưng dù nó xuất hiện ở đầu hay cuối câu — đó là tính bất biến đối với phép tịnh tiến. Hơn nữa, phép tích chập là tổng của các tích; nếu bộ lọc có các trọng số giống nhau thì đổi chỗ các từ trong cửa sổ không làm thay đổi kết quả (giống như $a + b + c = c + a + b$). Trong khi đó, với ngôn ngữ, "chó cắn người" và "người cắn chó" có nghĩa rất khác nhau, nên ta cần cách đưa thông tin vị trí vào mô hình một cách tường minh.

Mô hình hóa tường minh trật tự từ là rất quan trọng trong NLP, và đã được nghiên cứu rộng rãi trong các bài toán như dịch máy (*machine translation*) [Lopez, 2008; Koehn, 2010]. Với các mô hình dựa trên mạng

nơ-ron, người ta có thể giải quyết vấn đề bằng cách dùng các kiến trúc mô hình nhạy với thứ tự như RNN. Một cách tiếp cận phổ biến hơn trong các hệ thống gần đây là phát triển một mô hình con **mã hóa vị trí** (*positional encoding*) và tích hợp nó vào các mô hình chuỗi sẵn có [Gehring và cộng sự, 2017b; Vaswani và cộng sự, 2017; Shaw và cộng sự, 2018; Dufter và cộng sự, 2022]. Một cách hình thức, ta nói rằng đầu vào tại vị trí i là sự kết hợp giữa đầu vào gốc $\mathbf{x}_i$ và mã hóa vị trí của i (ký hiệu là $\text{PE}(\cdot)$):

$$\mathbf{e}_i = \text{Merge}(\mathbf{x}_i, \text{PE}(i)), \tag{4.46}$$

trong đó $\text{Merge}(\cdot)$ định nghĩa một phép kết hợp. Việc dùng mã hóa vị trí rất đơn giản: ta chỉ cần thay $\{\mathbf{x}_1, \ldots, \mathbf{x}_m\}$ bằng $\{\mathbf{e}_1, \ldots, \mathbf{e}_m\}$ trong một mô hình chuỗi. Vì vậy, cách tiếp cận này *không phụ thuộc vào mô hình* (*model-agnostic*).

Trong tiểu mục này, chúng tôi trình bày một số phiên bản của $\text{PE}(\cdot)$ và các cách kết hợp chúng với $\mathbf{x}_i$. Lưu ý rằng thảo luận dưới đây không chỉ dành riêng cho CNN. Chúng tôi xét nó ở đây vì mã hóa vị trí hữu ích cho các mô hình không nhạy với thứ tự của đầu vào, và CNN là một ví dụ tốt để thấy nó được sử dụng như thế nào [Gehring và cộng sự, 2017a;b]. Trong Chương 6, chúng ta sẽ thấy một ứng dụng của mã hóa vị trí trong Transformer, một mô hình nơ-ron tiên tiến nhất trong nhiều lĩnh vực.

1. Mã hóa vị trí dựa trên độ lệch

Cách đơn giản nhất để biểu diễn một vị trí i là dùng chỉ số tuyệt đối của nó. Điều này có thể được biểu diễn dưới dạng "khoảng cách" tới một điểm tham chiếu

$$\text{PE}(i) = i - i_0, \tag{4.47}$$

trong đó i_0 là một số nguyên cho biết ta bắt đầu đếm từ đâu. Nếu $i_0 = 0$ thì $\text{PE}(i) = i$ cho ta vị trí tuyệt đối tiêu chuẩn. Lưu ý rằng $\text{PE}(i)$ có thể là số âm nếu $i_0 > i$. Dù không phải là khoảng cách vật lý theo nghĩa chặt chẽ, nó vẫn là một đặc trưng hoàn toàn hợp lệ đối với các thuật toán học máy. Để bắt buộc đại lượng này không âm, vế phải của Phương trình (4.47) có thể được lấy giá trị tuyệt đối:

$$\text{PE}(i) = |i - i_0|. \tag{4.48}$$

Coi vị trí là các số nguyên đơn thuần dẫn tới một mã hóa vị trí rời rạc và không bị chặn. Một phương pháp đáng mong muốn hơn có thể là dùng

một biểu diễn liên tục trong một khoảng giá trị, vì nó cho phép hệ thống làm việc trong một không gian mẫu trơn và dễ tối ưu. Một cách đơn giản để làm điều này là chuẩn hóa. Ví dụ, chia $i - i_0$ cho một giá trị cực đại nào đó, ta được một phiên bản chuẩn hóa của mã hóa dựa trên độ lệch

$$\mathrm{PE}(i) = \frac{i - i_0}{i_{\max} - i_0}. \tag{4.49}$$

Ví dụ, ta có thể đặt $i_{\max}$ = độ dài lớn nhất có thể có của chuỗi và $i_0 = 0$, để $\mathrm{PE}(i)$ nhận giá trị trong $[0, 1]$. Một lựa chọn phổ biến khác là đặt $i_{\max} = m$ (tức là độ dài của chuỗi đầu vào) và định nghĩa $\mathrm{PE}(i)$ như một tỉ số có giá trị thay đổi khi m thay đổi.

Để tận dụng các vị trí dạng số vô hướng này, cách đơn giản là làm giàu các vectơ đầu vào gốc bằng cách thêm các chiều mới, miễn là chúng có thể được xem như các đặc trưng mới. Như vậy, $\mathbf{e}_i$ được cho bởi:

$$\mathbf{e}_i = [\mathbf{x}_i, \mathrm{PE}(i)], \tag{4.50}$$

trong đó $[\cdot]$ biểu thị phép ghép nối (*concatenation*).

Chú giải của người dịch: Ví dụ số

Chuỗi có $m = 4$ từ, chọn $i_0 = 0$, $i_{\max} = m = 4$. Theo Phương trình (4.49), các vị trí $1, 2, 3, 4$ được mã hóa thành 0.25, 0.5, 0.75, 1. Nếu $\mathbf{x}_2 = (0.3, -1.2)$ thì theo Phương trình (4.50), $\mathbf{e}_2 = (0.3, -1.2, 0.5)$: vectơ chỉ dài thêm một chiều chứa thông tin vị trí.

2. Mã hóa vị trí hình sin

Bước tiếp theo hiển nhiên là biểu diễn vị trí bằng vectơ thay vì số vô hướng. Mặc dù vectơ hóa biểu diễn vị trí nghe có vẻ phức tạp, một ý tưởng đơn giản là dùng một *hệ đếm có nhớ* (*carrying system*), mô tả cách một số tự nhiên được biểu diễn bằng một đa thức theo một cơ số [Kernes, 2021]. Ví dụ, i có thể được viết là

$$i = \sum_{k=0}^{k_{\max}} a(i,k)\, b^k, \tag{4.51}$$

trong đó $a(i,k)$ là chữ số thứ k, $k_{\max} + 1$ là số chữ số tối đa, và b là cơ số của hệ. Việc nhớ sang hàng tiếp theo xảy ra khi $a(i,k)$ đạt tới b: ta tăng

$a(i, k+1)$ thêm 1 và đưa $a(i, k)$ trở về 0. Theo cách này, ta có thể làm $a(i, k)$ thay đổi với chu kỳ b^k, tức là $a(i, 0)$ thay đổi với chu kỳ b^0, $a(i, 1)$ thay đổi với chu kỳ b^1, $a(i, 2)$ thay đổi với chu kỳ b^2, và cứ thế.

Dùng hệ này, i có thể được biểu diễn thành một vectơ

$$\text{PE}(i) = \begin{bmatrix} a(i,0) & a(i,1) & \cdots & a(i,k_{\text{max}}) \end{bmatrix}. \tag{4.52}$$

Ví dụ, khi $b = 2$ thì $\text{PE}(11) = \begin{bmatrix} 1 & 1 & 0 & 1 \end{bmatrix}$. Tuy nhiên, trong Phương trình (4.52), $\text{PE}(i)$ vẫn là một hàm rời rạc. Như đã bàn xuyên suốt cuốn sách này, ta có thể muốn có một biểu diễn vectơ liên tục, có khả năng mô tả các trạng thái trung gian giữa những sự kiện rời rạc. Coi $a(i, k)$ là một hàm tuần hoàn, một lựa chọn phổ biến là hàm **sin**. Như vậy, $a(i, k)$ có thể được định nghĩa lại như sau:

$$a(i, k) = \sin(i \cdot \omega_k). \tag{4.53}$$

Hàm này có biên độ bằng 1 và chu kỳ $\frac{2\pi}{\omega_k}$. Dùng một dạng chu kỳ tương tự như trong Phương trình (4.51), ta định nghĩa ω_k là

$$\omega_k = \frac{1}{(b_{\text{model}})^{k/d_{\text{model}}}}, \tag{4.54}$$

trong đó $b_{\text{model}} > 0$ và $d_{\text{model}} > 0$ là các siêu tham số của mô hình. Rõ ràng ta có $\frac{2\pi}{\omega_0} < \frac{2\pi}{\omega_1} < \cdots < \frac{2\pi}{\omega_{k_{\text{max}}}}$.

Chú giải của người dịch: Từ hệ nhị phân đến hàm sin

Viết số 11 trong hệ nhị phân: $11 = 1 \cdot 2^0 + 1 \cdot 2^1 + 0 \cdot 2^2 + 1 \cdot 2^3$, nên vectơ chữ số là $(1, 1, 0, 1)$ (đọc từ hàng đơn vị lên). Khi đếm $0, 1, 2, 3, \ldots$, chữ số hàng 2^0 đổi giá trị sau mỗi 1 bước, hàng 2^1 sau mỗi 2 bước, hàng 2^2 sau mỗi 4 bước… — hàng càng cao "dao động" càng chậm. Mã hóa hình sin giữ đúng ý tưởng này nhưng thay các chữ số nhảy bậc 0/1 bằng các sóng sin mượt, mỗi chiều k một sóng có chu kỳ $2\pi/\omega_k = 2\pi\,(b_{\text{model}})^{k/d_{\text{model}}}$ (nhớ lại ở lớp 11: hàm $\sin(\omega x)$ có chu kỳ $2\pi/\omega$). Vì $b_{\text{model}} > 1$ trong thực tế, k càng lớn thì chu kỳ càng dài.

Tương tự, ta có thể định nghĩa $a(i, k)$ thông qua hàm **cos**

$$a(i, k) = \cos(i \cdot \omega_k). \tag{4.55}$$

Kết hợp cả hai Phương trình (4.53) và (4.55), ta tạo ra một biểu diễn mới của i như sau:

$$\text{PE}(i) = \left[\sin(i \cdot \omega_0) \quad \cos(i \cdot \omega_0) \quad \cdots \quad \sin(i \cdot \omega_{k_{\max}}) \quad \cos(i \cdot \omega_{k_{\max}})\right]. \tag{4.56}$$

Vaswani và cộng sự [2017] đã cụ thể hóa dạng trên bằng cách đặt $b_{\text{model}} = 10{,}000$. Gọi $\text{PE}(i, k)$ là chiều thứ k của $\text{PE}(i)$. Phiên bản mã hóa vị trí của Vaswani và cộng sự [2017] được viết là

$$\text{PE}(i, 2k) = \sin\left(i \cdot \frac{1}{10000^{2k/d_{\text{model}}}}\right), \tag{4.57}$$

$$\text{PE}(i, 2k+1) = \cos\left(i \cdot \frac{1}{10000^{2k/d_{\text{model}}}}\right). \tag{4.58}$$

Việc chọn $b_{\text{model}} = 10{,}000$ là theo kinh nghiệm. Ta có thể điều chỉnh nó cho từng bài toán cụ thể. Hình 4.9 vẽ mã hóa vị trí cho các vị trí khác nhau. Ta thấy rằng khi k lớn dần, sự thay đổi màu sắc diễn ra theo chu kỳ dài hơn.

Lưu ý rằng Phương trình (6.14) và (6.15)[10] có một tính chất hữu ích là $\text{PE}(i + \mu)$ có thể dễ dàng được biểu diễn bằng một hàm tuyến tính của $\text{PE}(i)$ với một độ lệch μ cho trước[11]

$$\begin{aligned}\text{PE}(i + \mu, 2k) = \text{PE}(i, 2k) \cdot \text{PE}(\mu, 2k+1) + \\ \text{PE}(i, 2k+1) \cdot \text{PE}(\mu, 2k),\end{aligned} \tag{4.61}$$

$$\begin{aligned}\text{PE}(i + \mu, 2k+1) = \text{PE}(i, 2k+1) \cdot \text{PE}(\mu, 2k+1) - \\ \text{PE}(i, 2k) \cdot \text{PE}(\mu, 2k).\end{aligned} \tag{4.62}$$

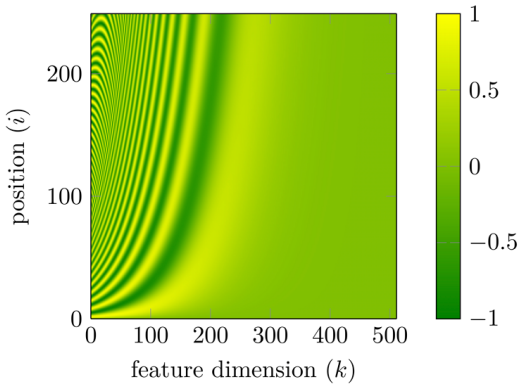
Tính chất này có lợi vì nó cho phép mô hình chú ý tới các vị trí tương đối. Nghĩa là, trạng thái tại vị trí $i + \mu$ có thể được mô tả bằng cách bắt đầu từ i rồi cộng thêm độ lệch μ.

[10] **[ND]** Đây chính là hai công thức (4.57) và (4.58) ở trên; chúng được trình bày lại trong Chương 6 với số hiệu (6.14) và (6.15).

[11] Ta có thể suy ra các công thức này bằng cách dùng

$$\sin(\alpha + \beta) = \sin(\alpha) \cdot \cos(\beta) + \cos(\alpha) \cdot \sin(\beta), \tag{4.59}$$

$$\cos(\alpha + \beta) = \cos(\alpha) \cdot \cos(\beta) - \sin(\alpha) \cdot \sin(\beta). \tag{4.60}$$



Hình 4.9: Bản đồ nhiệt của mô hình nhúng vị trí theo Phương trình (6.14) và (6.15) ($b_{\text{model}} = 10{,}000$ và $d_{\text{model}} = 512$). Xét một vị trí i (tức là hàng thứ i), rồi di chuyển một vị trí khác j từ i lên trên hoặc xuống dưới. Một cách trực quan, khi i và j càng gần nhau thì các vectơ hàng tương ứng càng giống nhau. Ngược lại, khi j di chuyển ra xa i, sự giống nhau không còn rõ rệt như vậy. Tính chất này giúp giải thích ý tưởng đằng sau mô hình nhúng vị trí: "khoảng cách" giữa hai vị trí được mô hình hóa một cách ngầm định bằng cách so sánh các biểu diễn nhiều chiều của chúng. (Trục tung: vị trí i; trục hoành: chiều đặc trưng k; thang màu từ -1 đến 1.)

Chú giải của người dịch: Công thức cộng lượng giác ở đây

Đặt $\omega = 1/10000^{2k/d_{\text{model}}}$. Khi đó $\text{PE}(i, 2k) = \sin(i\omega)$, $\text{PE}(i, 2k+1) = \cos(i\omega)$. Áp dụng công thức cộng lớp 11 với $\alpha = i\omega$, $\beta = \mu\omega$: $\sin((i+\mu)\omega) = \sin(i\omega)\cos(\mu\omega) + \cos(i\omega)\sin(\mu\omega)$ — đó chính là Phương trình (4.61). Điểm mấu chốt: với μ cố định, $\cos(\mu\omega)$ và $\sin(\mu\omega)$ chỉ là các hằng số, nên mã hóa của vị trí $i+\mu$ là một tổ hợp tuyến tính (nhân với hằng số rồi cộng) của mã hóa của vị trí i. Nhờ vậy mô hình dễ "học" được quan hệ kiểu "cách nhau μ vị trí".

Khi áp dụng mã hóa vị trí hình sin, một cách là dùng Phương trình (4.50) để ghép nối $\mathbf{x}_i$ và $\text{PE}(i)$. Trong công trình của Vaswani và cộng sự [2017], thay vào đó họ giả định $\text{PE}(i)$ là một vectơ có cùng kích thước với $\mathbf{x}_i$ (tức là $|\text{PE}(i)| = |\mathbf{x}_i| = d_e$), và cộng $\text{PE}(i)$ vào $\mathbf{x}_i$

$$\mathbf{e}_i = \mathbf{x}_i + \text{PE}(i). \tag{4.63}$$

Mô hình cộng dạng hình sin này đã là nền tảng của nhiều cách tiếp cận mã hóa vị trí [Dehghani và cộng sự, 2018; Likhomanenko và cộng sự, 2021; Su và cộng sự, 2021].

3. Mã hóa vị trí học được

Việc áp dụng mã hóa vị trí hình sin cho ta một bảng tra (*lookup table*) $\mathbf{C}_{\text{PE}} \in \mathbb{R}^{m_{\max} \times d_e}$ (trong đó $m_{\max}$ là độ dài chuỗi lớn nhất)

$$\mathbf{C}_{\text{PE}} = \begin{bmatrix} \text{PE}(1) \\ \vdots \\ \text{PE}(m_{\max}) \end{bmatrix}. \tag{4.64}$$

Trong bảng này, mỗi vectơ hàng $\text{PE}(i)$ tương ứng với vectơ nhúng của một vị trí i. Như đã mô tả ở trên, các vectơ hình sin này là tất định và được suy ra từ các hàm toán học cố định. Một cách tiếp cận khác là coi các vectơ vị trí là tham số của mô hình và học chúng như thường lệ. Trong trường hợp này, cả vectơ nhúng từ và vectơ nhúng vị trí đều có thể được huấn luyện theo cùng một cách. Xem Chương 2 và 3 để biết thêm về việc học vectơ nhúng từ trong các mô hình ngôn ngữ nơ-ron.

Một lưu ý cuối cùng về mã hóa vị trí. Những gì chúng tôi trình bày trong mục này nhìn chung có thể được xếp vào mô thức **mã hóa vị trí**

tuyệt đối (*absolute positional encoding*): một vị trí được mô tả bằng chỗ của nó trong một hệ tọa độ. Một khái niệm khác đáng tìm hiểu là **mã hóa vị trí tương đối** (*relative positional encoding*) [Shaw và cộng sự, 2018]. Ví dụ, ta có thể mở rộng Phương trình (4.48) để định nghĩa khoảng cách giữa hai vị trí i và j

$$\mathrm{PE}(i, j) = |i - j|. \tag{4.65}$$

Trong trường hợp này, mã hóa vị trí không còn là một thuộc tính của đầu vào thứ i, mà là một biểu diễn nào đó của khoảng cách so với một vị trí tham chiếu j. Thực ra, hầu hết các phương pháp mã hóa vị trí tương đối đều là các biến thể xoay quanh một chủ đề chung, trong đó các vị trí được mô tả bằng quan hệ theo từng cặp của chúng. Thiên kiến quy nạp mang tính quan hệ này tạo nên nền tảng của một số kiến trúc tiên tiến, sẽ được trình bày chi tiết trong Chương 6 và 8.

Chú giải của người dịch: Tuyệt đối và tương đối

Ví dụ đời thường: "nhà tôi ở số 25 phố Huế" là cách mô tả *tuyệt đối*; "nhà tôi cách nhà bạn 3 căn" là cách mô tả *tương đối*. Với ngôn ngữ, điều quan trọng thường là quan hệ giữa hai từ (tính từ đứng ngay sau danh từ, động từ cách chủ ngữ 2 từ…) hơn là việc một từ đứng ở vị trí thứ 37 trong văn bản; đó là lý do mã hóa vị trí tương đối được quan tâm nhiều.

4.5 Các ví dụ

Cả mô hình hồi quy và mô hình tích chập đều đã được dùng thành công trong rất nhiều ứng dụng. Ở đây chúng ta bàn về một vài ví dụ thú vị. Dù các mô hình này phần lớn là cơ bản, chúng tạo nền móng cho nhiều hệ thống tiên tiến nhất.

4.5.1 Phân loại văn bản

Để minh họa cách dùng các mô hình chuỗi, trước hết ta xét bài toán phân loại văn bản (*text classification*), trong đó ta gán một lớp định sẵn cho một chuỗi văn bản cho trước. Khung bài toán này mở rộng một cách tự nhiên sang rất nhiều bài toán NLP, bao gồm phân loại bài báo, phân tích cảm xúc, nhận diện thư rác, phát hiện bình luận lừa đảo, v.v.

Trong phân loại văn bản, ta quan tâm tới việc dự đoán nhãn lớp từ một tập C, khi cho trước một chuỗi từ $w_1 \cdots w_m$:

$$\hat{c} = \underset{c \in C}{\arg\max}\ \text{Score}(c, w_1 \cdots w_m). \tag{4.66}$$

Ở đây $\text{Score}(c, w_1 \cdots w_m)$ đo mức độ phù hợp khi dự đoán lớp c cho chuỗi đầu vào $w_1 \cdots w_m$. Ta ánh xạ chuỗi từ thành chuỗi vectơ từ (hay vectơ nhúng từ), tức là $w_1 \cdots w_m \rightarrow \mathbf{x}_1 \cdots \mathbf{x}_m$. Giả sử $\text{Score}(\cdot)$ là một hàm xác suất mô tả phân phối của các lớp, ta có thể phát biểu lại bài toán thành

$$\begin{aligned} \hat{c} &= \underset{c \in C}{\arg\max}\ \Pr(c | w_1 \cdots w_m) \\ &= \underset{c \in C}{\arg\max}\ \Pr(c | \mathbf{x}_1 \cdots \mathbf{x}_m). \end{aligned} \tag{4.67}$$

Vấn đề trung tâm ở đây là mô hình hóa $\Pr(c|\mathbf{x}_1 \cdots \mathbf{x}_m)$. Ta định nghĩa $\Pr(c|\mathbf{x}_1 \cdots \mathbf{x}_m)$ theo khung *bộ mã hóa – bộ dự đoán* tổng quát như sau

- Đầu vào $\mathbf{x}_1 \cdots \mathbf{x}_m$ được biểu diễn thành một vectơ đặc trưng $\mathbf{H} \in \mathbb{R}^{d_h}$ nhờ một bộ mã hóa (*encoder*)

$$\mathbf{H} = \text{Encode}(\mathbf{x}_1 \cdots \mathbf{x}_m). \tag{4.68}$$

- $\mathbf{H}$ được đưa vào một tầng Softmax tiêu chuẩn để dự đoán phân phối các lớp

$$\Pr(\cdot|\mathbf{H}) = \text{Softmax}(\mathbf{H} \cdot \mathbf{U} + \mathbf{b}), \tag{4.69}$$

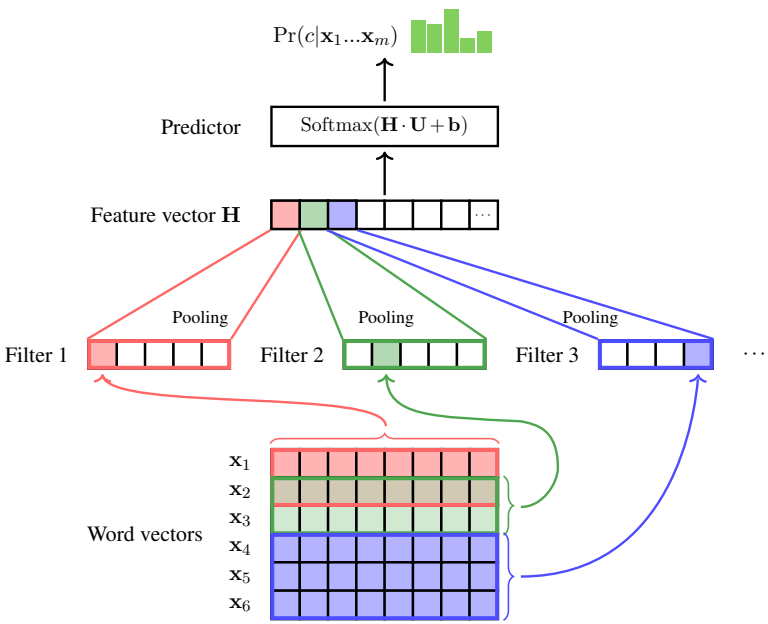
trong đó $\mathbf{U} \in \mathbb{R}^{d_h \times |C|}$ và $\mathbf{b} \in \mathbb{R}^{|C|}$ là các tham số của mô hình.

Chú giải của người dịch: Ví dụ phân loại cảm xúc

Giả sử $C = \{\text{tích cực, tiêu cực}\}$, nên $|C| = 2$. Bộ mã hóa nén cả câu "Phim này hay quá" thành một vectơ $\mathbf{H}$ (chẳng hạn $d_h = 100$ số). Nhân với $\mathbf{U}$ (kích thước 100×2) rồi cộng $\mathbf{b}$ cho ra 2 "điểm số", ví dụ $(2.0,\ 0.0)$. Softmax biến chúng thành xác suất: $e^2/(e^2 + e^0) \approx 0.88$ cho "tích cực" và ≈ 0.12 cho "tiêu cực". Phép arg max trong Phương trình (4.67) đơn giản là chọn lớp có xác suất lớn nhất: "tích cực".

Thành phần $\text{Encode}(\cdot)$ giống như các kiến trúc đã bàn ở những mục trước. Do đó, rất nhiều mô hình mã hóa chuỗi có thể được áp dụng trực

tiếp ở đây. Ví dụ, xét bộ mã hóa dựa trên CNN được trình bày trong công trình của Kim [2014]. Mô hình của Kim [2014] dựa trên một tầng tích chập duy nhất gồm d_h bộ lọc. Việc áp dụng một bộ lọc tạo ra một tập đặc trưng, mỗi đặc trưng gắn với một vị trí của chuỗi. Vì ta muốn có một vectơ duy nhất để biểu diễn toàn bộ chuỗi, một tầng gộp được thêm vào để số đặc trưng của mỗi bộ lọc giảm xuống còn một. Sau đó, với mọi c, xác suất $\Pr(c|\mathbf{H})$ có thể được tính trực tiếp theo Phương trình (4.69). Xem Hình 4.10 để thấy minh họa của bộ phân loại này.



Hình 4.10: Một bộ phân loại văn bản dựa trên CNN [Kim, 2014]. Đầu vào là một chuỗi vectơ từ (*Word vectors*). Một tầng tích chập gồm nhiều bộ lọc (*Filter*) được dùng để trích xuất đặc trưng theo các chiều khác nhau. Một tầng gộp (*Pooling*) được dùng để giảm số lượng đặc trưng biểu diễn văn bản đầu vào, dẫn tới một vectơ đặc trưng (*Feature vector*) $\mathbf{H}$ có số chiều thấp. Việc dự đoán được thực hiện có điều kiện trên $\mathbf{H}$ bằng một tầng Softmax tiêu chuẩn (*Predictor*: bộ dự đoán).

Huấn luyện một mô hình như vậy bao gồm việc tối ưu một hàm mất mát định sẵn, và như đã nhắc nhiều lần trong sách, một trong những

phương pháp phổ biến nhất là cực tiểu hóa mất mát entropy chéo bằng hạ gradient. Ngoài ra, ta có thể dùng chính quy hóa để cải thiện việc huấn luyện CNN. Chi tiết về các kỹ thuật này có thể xem ở Chương 2.

Đáng lưu ý là dù mô hình cơ sở này đơn giản, nó vẫn là một trong những kiến trúc hiệu quả nhất cho phân loại văn bản. Có nhiều cải tiến kiến trúc. Các mở rộng nổi bật của khung này gồm CNN sâu [Conneau và cộng sự, 2017c], CNN mức ký tự [Santos và Gatti, 2014; Zhang và cộng sự, 2015], và các mô hình lai hồi quy–tích chập [Lai và cộng sự, 2015].

4.5.2 Nhận dạng tiếng nói đầu-cuối

Nhận dạng tiếng nói (*speech recognition*) là bài toán ánh xạ một chuỗi tín hiệu âm thanh thành một chuỗi từ hoặc ký tự (gọi là **bản phiên âm** (*transcription*)) [Reddy, 1976; Rabiner và Juang, 1993]. Vì đầu vào gốc là một dạng sóng âm theo miền thời gian, cách làm tiêu chuẩn là biến đổi nó thành một chuỗi các đoạn sóng (gọi là **khung** (*frames*)). Thông thường, một khung được biểu diễn bằng một vectơ đặc trưng, ký hiệu là $\mathbf{x}_i$. Điều này được thực hiện bằng cách dùng hoặc các hàm đặc trưng được thiết kế thủ công từ lĩnh vực xử lý tín hiệu [Davis và Mermelstein, 1980; Picone, 1993; Campbell, 1997], hoặc các vectơ nhúng nơ-ron học được [Chorowski và cộng sự, 2019; Schneider và cộng sự, 2019]. Về phía đầu ra, các hệ thống nhận dạng tiếng nói nói chung không trực tiếp sinh ra các từ hoàn chỉnh. Thay vào đó, chúng xuất ra chuỗi các **đơn vị phiên âm** (*transcription units*) (hay **nhãn phiên âm** (*transcription labels*)), chẳng hạn âm vị, ký tự hoặc từ con. Trong tiểu mục này, ta giả định đầu ra của hệ thống là một chuỗi ký tự, ký hiệu là $y_1, \ldots, y_n \in V_y^n$. Bộ từ vựng V_y gồm các chữ cái tiếng Anh tiêu chuẩn (a − z), chữ số (0 − 9), dấu cách ($\langle$sp$\rangle$), dấu chấm ($\langle$pe$\rangle$) và các dấu câu khác. Giống như hầu hết các hệ thống nhận dạng tiếng nói hiện đại, ta thêm một ký hiệu trống ϵ vào bảng chữ cái để biểu thị đầu ra rỗng.

Mục tiêu ở đây là tìm một chuỗi ký tự $\hat{y}_1 \cdots \hat{y}_n$ cho một chuỗi đầu vào $\mathbf{x}_1 \cdots \mathbf{x}_m$ cho trước, sao cho

$$\hat{y}_1 \cdots \hat{y}_n = \underset{y_1 \cdots y_n}{\arg\max} \Pr(y_1 \cdots y_n | \mathbf{x}_1 \cdots \mathbf{x}_m). \tag{4.70}$$

Bài toán này khó hơn bài toán phân loại ở tiểu mục trước, vì đầu ra có thể là một chuỗi ký tự bất kỳ chứ không phải một lớp trong một tập lớp

định sẵn. Bài toán sinh chuỗi dẫn tới hai khó khăn. Thứ nhất, ta cần một cơ chế nào đó để mô hình hóa $\Pr(y_1 \cdots y_n | \mathbf{x}_1 \cdots \mathbf{x}_m)$ cho một số lượng cặp chuỗi đầu vào–đầu ra lớn theo cấp số mũ. Thứ hai, trong thực tế $y_1 \cdots y_n$ thường ngắn hơn nhiều so với $\mathbf{x}_1 \cdots \mathbf{x}_m$ (tức là $n < m$), nên ta cần một cơ chế để gióng một chuỗi dài với một chuỗi ngắn. Tuy nhiên, ta không cần xét các khó khăn này ở giai đoạn biểu diễn chuỗi đầu vào, và vẫn có thể mã hóa chuỗi đầu vào $\mathbf{x}_1 \cdots \mathbf{x}_m$ theo cùng cách như các mô hình chuỗi khác. Cụ thể, ta biểu diễn $\mathbf{x}_1 \cdots \mathbf{x}_m$ dưới dạng

$$\mathbf{h}_1 \cdots \mathbf{h}_m = \text{Encode}(\mathbf{x}_1 \cdots \mathbf{x}_m). \tag{4.71}$$

Bộ mã hóa có thể là RNN, CNN, hoặc các mô hình tiên tiến hơn (như Transformer). Ở đây ta xét kiến trúc bộ mã hóa được dùng trong công trình của Graves và cộng sự [2013b] và Graves và Jaitly [2014]. Đó là một mô hình LSTM hai chiều nhiều tầng. Chúng tôi bỏ qua chi tiết của mô hình này mà không làm mất mạch trình bày, vì bạn đọc có lẽ đã quen với nó qua các Mục 4.2.3, 4.2.4 và 4.3.2.

Sau khi có biểu diễn chuỗi $\mathbf{H} = \mathbf{h}_1 \cdots \mathbf{h}_m$, một tầng Softmax được dùng để ánh xạ mỗi $\mathbf{h}_i \in \mathbb{R}^{d_h}$ thành một phân phối trên các nhãn phiên âm, cho bởi:

$$\Pr(\cdot | \mathbf{h}_i) = \text{Softmax}(\mathbf{h}_i \cdot \mathbf{U} + \mathbf{b}), \tag{4.72}$$

trong đó $\mathbf{U} \in \mathbb{R}^{d_h \times |V_y|}$ và $\mathbf{b} \in \mathbb{R}^{|V_y|}$ là các tham số mô hình. $\Pr(\cdot | \mathbf{h}_i) \in \mathbb{R}^{|V_y|}$ là một phân phối xác suất trên V_y, và xác suất của nhãn phiên âm l_i tại vị trí i đơn giản là $\Pr(l_i | \mathbf{h}_i)$. Khi đó ta có thể viết xác suất của một chuỗi nhãn dưới dạng

$$\Pr(l_1 \cdots l_m | \mathbf{H}) = \prod_{i=1}^{m} \Pr(l_i | \mathbf{h}_i). \tag{4.73}$$

Ta có thể áp dụng phép arg max để tìm chuỗi nhãn có xác suất cao nhất như thường lệ. Tuy nhiên, $l_1 \cdots l_m$ không thể dùng trực tiếp làm đầu ra của hệ thống, vì nó thường chứa nhiều ký hiệu trùng lặp và ký hiệu trống. Để hậu xử lý $l_1 \cdots l_m$, trước hết ta gộp các nhãn giống nhau liên tiếp thành một nhãn duy nhất, rồi xóa các ký hiệu trống. Ví dụ, xét chuỗi nhãn

$$\text{s s } \epsilon \text{ e e } \epsilon \text{ } \epsilon \text{ e } \langle\text{sp}\rangle \text{ y } \epsilon \text{ o o u.}$$

Bằng cách gộp "s s", "e e" và "o o", ta có

$$\text{s } \epsilon \text{ e } \epsilon \ \epsilon \text{ e } \langle\text{sp}\rangle \text{ y } \epsilon \text{ o u.}$$

Sau đó, ta xóa mọi ϵ và thu được

$$\text{s e e } \langle\text{sp}\rangle \text{ y o u.}$$

Chuỗi trên chính là cái mà ta gọi là một bản phiên âm ("see you" — "hẹn gặp lại").

Chú giải của người dịch: Vì sao cần ký hiệu trống ϵ?

Âm thanh được cắt thành rất nhiều khung (ví dụ 100 khung mỗi giây), nên một chữ thường kéo dài qua nhiều khung liên tiếp: "s s" chỉ là một chữ s được nói kéo dài. Vì vậy ta gộp các nhãn lặp liền nhau. Nhưng từ "see" có hai chữ e đứng cạnh nhau thật sự; nếu chỉ gộp thì "e e e" sẽ thành một chữ e duy nhất. Ký hiệu trống ϵ đóng vai trò "vách ngăn": "e ϵ e" sau khi gộp và xóa ϵ vẫn giữ được hai chữ e. Ngoài ra, ϵ còn dùng cho những khung im lặng hoặc không ứng với chữ nào.

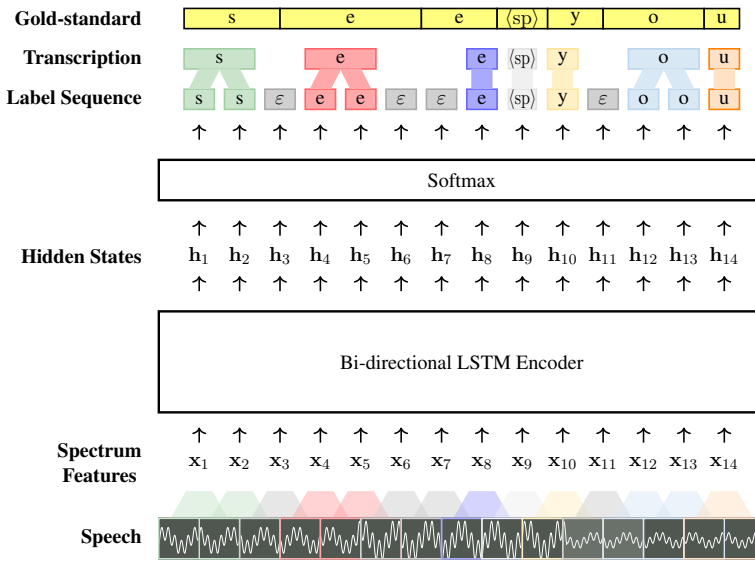
Rõ ràng, các chuỗi nhãn khác nhau có thể ứng với cùng một bản phiên âm. Gọi $B(y_1 \cdots y_n)$ là tập các chuỗi nhãn ứng với $y_1 \cdots y_n$[12]. Bây giờ ta chuyển sang dạng sau của xác suất phiên âm (xem minh họa ở Hình 4.11)

$$\Pr(y_1 \cdots y_n | \mathbf{x}_1 \cdots \mathbf{x}_m) = \sum_{l_1 \cdots l_m \in B(y_1 \cdots y_n)} \Pr(l_1 \cdots l_m | \mathbf{H}). \tag{4.74}$$

Một vấn đề của mô hình này là số chuỗi trong $B(y_1 \cdots y_n)$ tăng theo cấp số mũ với n (và m). May mắn là có những phương pháp rất hiệu quả để tính $\sum_{l_1 \cdots l_m \in B(y_1 \cdots y_n)} \Pr(l_1 \cdots l_m | \mathbf{H})$. Xem [Graves và cộng sự, 2006] để biết một thuật toán quy hoạch động (*dynamic programming*) giải bài toán này.

Lưu ý rằng Phương trình (4.74) còn được biết đến như một dạng của **phân loại thời gian kết nối** (*connectionist temporal classification*) (**CTC**) [Graves và cộng sự, 2006]. Đây là một trong những phương pháp được dùng rộng rãi nhất để huấn luyện các hệ thống nhận dạng tiếng nói và dịch tiếng nói đầu-cuối. Một ưu điểm của CTC là nó cho phép gióng bất

[12] $B(y_1 \cdots y_n)$ có thể chứa các chuỗi nhãn có độ dài bất kỳ. Tuy nhiên, nếu ta giới hạn đầu vào là chuỗi $\mathbf{x}_1 \cdots \mathbf{x}_m$ thì mỗi chuỗi trong $B(y_1 \cdots y_n)$ có độ dài m.



Hình 4.11: Một kiến trúc nhận dạng tiếng nói đầu-cuối. Đầu vào của hệ thống là một chuỗi tín hiệu âm thanh (*Speech*) được biểu diễn thành một chuỗi vectơ đặc trưng (*Spectrum Features*: đặc trưng phổ, tức là $\mathbf{x}_1 \cdots \mathbf{x}_{14}$). Các vectơ đặc trưng này được đưa vào một bộ mã hóa LSTM hai chiều (*Bi-directional LSTM Encoder*). Đầu ra của bộ mã hóa là một chuỗi các biểu diễn theo ngữ cảnh (*Hidden States*: trạng thái ẩn, tức là $\mathbf{h}_1 \cdots \mathbf{h}_{14}$), sau đó được đưa vào một tầng Softmax để sinh một phân phối nhãn tại mỗi vị trí. Từ các phân phối này, ta có thể sinh ra một chuỗi nhãn (*Label Sequence*). Ta ánh xạ mỗi chuỗi nhãn thành một dạng đầu ra cuối cùng (*Transcription*: bản phiên âm) bằng cách loại bỏ các ký hiệu trùng lặp và ký hiệu trống. Một đầu ra của hệ thống ứng với nhiều chuỗi nhãn, và xác suất của đầu ra là tổng xác suất của các chuỗi nhãn này. (*Gold-standard*: bản phiên âm chuẩn.)

kỳ chuỗi nhãn nào với một bản phiên âm theo cách rất đơn giản và hiệu quả. Việc dùng CTC để huấn luyện một hệ thống nhận dạng tiếng nói khá dễ dàng. Giả sử có một tập các cặp gồm chuỗi đầu vào và bản phiên âm, ký hiệu là $\mathcal{D}$. Một mục tiêu huấn luyện phổ biến là cực đại hóa hợp lý của các bản phiên âm này khi biết các đầu vào tương ứng, được viết là

$$\hat{\theta} = \arg\max_{\theta} \sum_{(y_1 \cdots y_n, \mathbf{x}_1 \cdots \mathbf{x}_m) \in \mathcal{D}} \log \Pr(y_1 \cdots y_n | \mathbf{x}_1 \cdots \mathbf{x}_m, \theta), \quad (4.75)$$

trong đó $\Pr(y_1 \cdots y_n | \mathbf{x}_1 \cdots \mathbf{x}_m, \theta)$ là xác suất được tính qua Phương trình (4.74), và θ là các tham số mô hình.

Chú giải của người dịch: Một ví dụ nhỏ về CTC

Giả sử chỉ có $m = 3$ khung và bản phiên âm là "a". Các chuỗi nhãn dài 3 thu gọn được thành "a" gồm: a a a, a a ϵ, a ϵ ϵ, ϵ a a, ϵ a ϵ, ϵ ϵ a — tổng cộng 6 chuỗi (lưu ý a ϵ a thu gọn thành "aa" nên không tính). Theo Phương trình (4.74), Pr("a") là *tổng* xác suất của 6 chuỗi này, mỗi chuỗi có xác suất là tích các xác suất theo từng khung (Phương trình (4.73)). Ta không cần biết trước chữ "a" được nói ở khung nào — mô hình tự "gióng" bằng cách cộng mọi khả năng. Khi m lớn, số khả năng tăng rất nhanh nên cần quy hoạch động để tính tổng mà không phải liệt kê từng chuỗi.

Khi kiểm tra trên dữ liệu mới, ta tìm một bản phiên âm tối ưu như trong Phương trình (4.70). Quá trình này, còn gọi là **giải mã** (*decoding*), nói chung cần các thuật toán tìm kiếm được tối ưu hóa và các kỹ thuật cắt tỉa. Ví dụ, ta có thể dùng chuỗi nhãn tốt nhất (1-best) thay vì mọi chuỗi nhãn có thể để xấp xỉ Phương trình (4.74), tức là $\Pr(y_1 \cdots y_n | \mathbf{x}_1 \cdots \mathbf{x}_m) = \max \Pr(l_1 \cdots l_m | \mathbf{H})$. Điều này dẫn tới một phương pháp giải mã hiệu quả, gọi là **giải mã Viterbi** (*Viterbi decoding*), đã được dùng rộng rãi trong nhận dạng tiếng nói và dịch máy [Lopez, 2008]. Để biết thêm chi tiết về giải mã trong sinh chuỗi, mời bạn đọc xem Chương 5.

4.5.3 Gán nhãn chuỗi với LSTM và mô hình đồ thị

Gán nhãn chuỗi (*sequence labeling*) là một cách tiếp cận đơn giản về mặt khái niệm để gán nhãn cho dữ liệu dạng chuỗi. Trong NLP, nó đã được dùng rộng rãi trong nhiều lĩnh vực con, như tách từ, gán nhãn từ loại và phân đoạn cụm (*chunking*). Việc học trong các mô hình này gồm việc dự

đoán một nhãn từ một tập nhãn định sẵn V_y tại mỗi vị trí của chuỗi. Lý tưởng nhất, ta muốn dự đoán chuỗi nhãn tối ưu khi biết toàn bộ đầu vào, được phát biểu là:

$$\hat{y}_1 \cdots \hat{y}_m = \underset{y_1 \cdots y_m}{\arg\max} \Pr(y_1 \cdots y_m | \mathbf{x}_1 \cdots \mathbf{x}_m), \tag{4.76}$$

trong đó $\mathbf{x}_1 \cdots \mathbf{x}_m$ là một chuỗi đầu vào (chẳng hạn một chuỗi vectơ từ), và $y_1 \cdots y_m$ là một chuỗi nhãn, trong đó mỗi nhãn y_i ứng với một phần tử đầu vào $\mathbf{x}_i$.

Chú giải của người dịch: Ví dụ gán nhãn từ loại

Với câu "Tôi đọc sách", đầu vào là 3 vectơ từ và đầu ra mong muốn là chuỗi nhãn "đại từ – động từ – danh từ": mỗi từ đúng một nhãn, nên chuỗi nhãn dài bằng chuỗi đầu vào (khác với nhận dạng tiếng nói ở tiểu mục trước, nơi $n < m$).

Như ta đã thấy trong chương này, Phương trình (4.76) rất khớp với cách phát biểu bài toán mô hình hóa chuỗi. Bước đầu tiên, ta dùng một bộ mã hóa để ánh xạ $\mathbf{x}_1 \cdots \mathbf{x}_m$ thành một chuỗi biểu diễn theo ngữ cảnh như sau:

$$\mathbf{h}_1 \cdots \mathbf{h}_m = \text{Encode}(\mathbf{x}_1 \cdots \mathbf{x}_m). \tag{4.77}$$

Ta định nghĩa Encode$(\cdot)$ là một mô hình LSTM hai chiều vì nó có cơ chế nhớ để mô hình hóa các phụ thuộc tầm xa ở cả ngữ cảnh bên trái lẫn bên phải. Do đó, kiến trúc của bộ mã hóa giống như kiến trúc dùng ở tiểu mục trước.

$\mathbf{h}_1 \cdots \mathbf{h}_m$ sau đó có thể làm đầu vào cho một hệ thống gán nhãn chuỗi tiêu chuẩn (xem Hình 4.12). Bài toán gán nhãn chuỗi đã được bàn ở Chương 1, và nhiều mô hình có thể được áp dụng để giải nó. Đơn giản nhất là một bộ phân loại (như mô hình entropy cực đại hay các mô hình dựa trên SVM) dự đoán một phân phối nhãn cho mỗi $\mathbf{h}_i$. Hạn chế của các mô hình này là các dự đoán được đưa ra độc lập với nhau. Một cách tiếp cận mạnh hơn là dùng **mô hình đồ thị** (*graphical models*) để tính đến sự phụ thuộc giữa các nhãn được dự đoán. Ví dụ, **mô hình Markov ẩn** (*hidden Markov models*) (**HMM**) mô tả cách một chuỗi quan sát (tức là $\mathbf{x}_1 \cdots \mathbf{x}_m$) được sinh ra khi cho trước một chuỗi biến (tức là $y_1 \cdots y_m$). Ý tưởng then chốt là viết lại $\Pr(y_1 \cdots y_m | \mathbf{x}_1 \cdots \mathbf{x}_m)$ bằng cách đưa ra các giả định độc lập để phân tích phân phối thành nhân tử, và xấp xỉ $\Pr(y_1 \cdots y_m | \mathbf{x}_1 \cdots \mathbf{x}_m)$

bằng một tích của các nhân tử đơn giản. Tuy nhiên, các mô hình này đòi hỏi hàm mật độ xác suất của các biến liên tục (ví dụ $\Pr(\mathbf{x}_i|y_i)$), vốn khó ước lượng. Điều này làm việc dùng HMM trong các mô hình nơ-ron khác biệt đáng kể so với trong các mô hình truyền thống, nơi mọi trạng thái và quan sát đều là biến rời rạc[13].

Chú giải của người dịch: Ý tưởng của HMM qua một ví dụ

Phương trình (4.78) dùng *định lý Bayes*: $\Pr(A|B) = \Pr(A)\Pr(B|A)/\Pr(B)$, cho phép "lật" chiều của xác suất có điều kiện ($\Pr(A|B)$ là xác suất xảy ra A khi đã biết B xảy ra). HMM thêm hai giả định đơn giản hóa: (1) nhãn hiện tại chỉ phụ thuộc vào nhãn ngay trước nó, $\Pr(y_i|y_{i-1})$ — ví dụ sau "động từ" thì khả năng gặp "danh từ" khá cao; (2) mỗi từ chỉ phụ thuộc vào nhãn của chính nó, $\Pr(\mathbf{x}_i|y_i)$ — ví dụ nhãn "động từ" hay "phát ra" từ "đọc". Nhờ đó, xác suất cả chuỗi trở thành tích của nhiều thừa số nhỏ, dễ ước lượng.

HMM và các mô hình hậu duệ của nó có thể được xem là các trường hợp của mô hình sinh. Một lớp mô hình khác thường được dùng cho gán nhãn chuỗi là mô hình phân biệt. Một mô hình như vậy là **trường ngẫu**

[13]Trong HMM, một chuỗi biến có thể được xem như một đường đi chuyển qua một số trạng thái, với giá trị được chọn từ một tập định sẵn. Trong mỗi lần chuyển từ trạng thái này sang trạng thái khác, ta quan sát được một thứ gì đó (gọi là một quan sát). Bằng cách đưa ra một số giả định, ta có thể xấp xỉ $\Pr(y_1 \cdots y_m|\mathbf{x}_1 \cdots \mathbf{x}_m)$ theo cách sau

$$\begin{aligned}\Pr(y_1 \cdots y_m|\mathbf{x}_1 \cdots \mathbf{x}_m) &= \frac{\Pr(y_1 \cdots y_m) \cdot \Pr(\mathbf{x}_1 \cdots \mathbf{x}_m|y_1 \cdots y_m)}{\Pr(\mathbf{x}_1 \cdots \mathbf{x}_m)} \\ &\approx \frac{\prod_{i=1}^m \Pr(y_i|y_{i-1}) \cdot \prod_{i=1}^m \Pr(\mathbf{x}_i|y_i)}{\Pr(\mathbf{x}_1 \cdots \mathbf{x}_m)} \\ &= \frac{\prod_{i=1}^m \Pr(y_i|y_{i-1}) \Pr(\mathbf{x}_i|y_i)}{\Pr(\mathbf{x}_1 \cdots \mathbf{x}_m)},\end{aligned} \tag{4.78}$$

trong đó $\Pr(y_i|y_{i-1})$ là **xác suất chuyển** (*transition probability*) từ y_{i-1} sang y_i (khi $i = 1$, ta định nghĩa $\Pr(y_i|y_{i-1}) = \Pr(y_1|y_0) = \Pr(y_1)$), và $\Pr(\mathbf{x}_i|y_i)$ là **xác suất phát xạ** (*emission probability*) của việc quan sát được $\mathbf{x}_i$ khi biết y_i. Vì mẫu số $\Pr(\mathbf{x}_1 \cdots \mathbf{x}_m)$ là hằng số đối với các $y_1 \cdots y_m$ khác nhau, nó có thể được bỏ đi trong phép argmax của Phương trình (4.76), như sau:

$$\hat{y}_1 \cdots \hat{y}_m = \operatorname*{arg\,max}_{y_1 \cdots y_m} \prod_{i=1}^m \Pr(y_i|y_{i-1}) \Pr(\mathbf{x}_i|y_i). \tag{4.79}$$

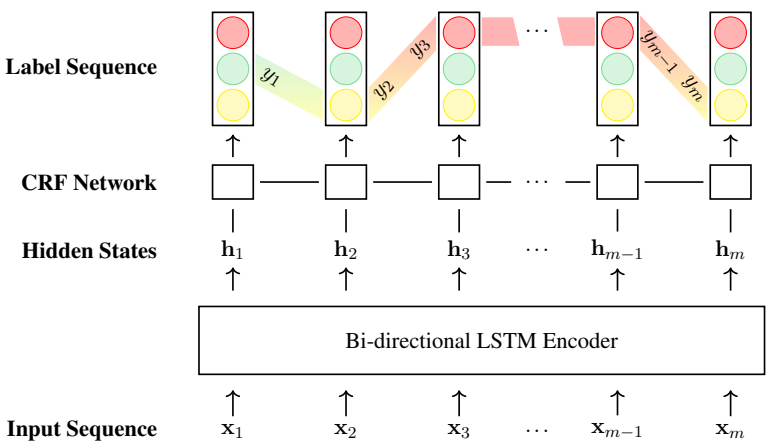
Để ước lượng $\Pr(\mathbf{x}_i|y_i)$, một giải pháp khả dĩ là lấy $\Pr(\mathbf{x}_i|y_i) = \frac{\Pr(y_i|\mathbf{x}_i)\Pr(\mathbf{x}_i)}{\Pr(y_i)}$, và dùng một mạng nơ-ron để tính $\Pr(y_i|\mathbf{x}_i)$.

nhiên có điều kiện (*conditional random fields*) (**CRF**) [Lafferty và cộng sự, 2001]. Đặc trưng của mô hình CRF là khả năng mô hình hóa trực tiếp xác suất có điều kiện của một chuỗi nhãn khi biết chuỗi đầu vào, điều này giúp dễ dàng tích hợp các đặc trưng tùy ý. Lấy ví dụ **CRF chuỗi tuyến tính** (*linear-chain CRF*) [Sutton và McCallum, 2012]. Nó định nghĩa $\Pr(y_1 \cdots y_m | \mathbf{h}_1 \cdots \mathbf{h}_m)$ dưới dạng sau

$$\begin{aligned}\Pr(y_1 \cdots y_m | \mathbf{h}_1 \cdots \mathbf{h}_m) &= \frac{\Pr(y_1 \cdots y_m, \mathbf{h}_1 \cdots \mathbf{h}_m)}{\Pr(\mathbf{h}_1 \cdots \mathbf{h}_m)} \\ &= \frac{\exp(\text{Score}(y_1 \cdots y_m, \mathbf{h}_1 \cdots \mathbf{h}_m))}{Z(\mathbf{h}_1 \cdots \mathbf{h}_m)}, \end{aligned} \tag{4.80}$$

trong đó $Z(\mathbf{h}_1 \cdots \mathbf{h}_m)$ là một hệ số chuẩn hóa, có dạng

$$Z(\mathbf{h}_1 \cdots \mathbf{h}_m) = \sum_{y'_1 \cdots y'_m} \exp(\text{Score}(y'_1 \cdots y'_m, \mathbf{h}_1 \cdots \mathbf{h}_m)). \tag{4.81}$$



Hình 4.12: Kiến trúc BiLSTM + mô hình đồ thị cho gán nhãn chuỗi. Bộ mã hóa là một mô hình LSTM hai chiều tiêu chuẩn (*Bi-directional LSTM Encoder*). Cho một chuỗi vectơ đặc trưng đầu vào (*Input Sequence*, tức là $\mathbf{x}_1 \cdots \mathbf{x}_m$), nó tạo ra một chuỗi vectơ đặc trưng mới để ánh xạ đầu vào thành các biểu diễn theo ngữ cảnh (*Hidden States*, tức là $\mathbf{h}_1 \cdots \mathbf{h}_m$). Một mạng CRF (*CRF Network*) được đặt trên $\mathbf{h}_1 \cdots \mathbf{h}_m$ để dự đoán một phân phối trên các chuỗi nhãn (*Label Sequence*). Chuỗi nhãn tối ưu là chuỗi có xác suất lớn nhất.

Score$(\cdot)$ là một điểm số dùng để đánh trọng số cho cặp chuỗi $(y_1 \cdots y_m, \mathbf{h}_1 \cdots \mathbf{h}_m)$. Nó được tính bằng cách lấy tổng giá trị của một tập hàm đặc trưng (*feature functions*) $\{f_1(\cdot), \ldots, f_J(\cdot)\}$

$$\text{Score}(y_1 \cdots y_m, \mathbf{h}_1 \cdots \mathbf{h}_m) = \sum_{i=1}^{m} \sum_{j=1}^{J} f_j(y_i, y_{i-1}, \mathbf{h}_i). \tag{4.82}$$

Vòng lặp ngoài của tổng ứng với việc duyệt qua từng vị trí i. Với mỗi i, mỗi hàm $f_j(\cdot)$ nhận nhãn hiện tại y_i, nhãn trước đó y_{i-1} và vectơ đầu vào hiện tại $\mathbf{h}_i$, rồi trả về giá trị của một đặc trưng.

Mô hình này được gọi là *chuỗi tuyến tính* vì nó giả định tính chất Markov bậc nhất: mỗi nhãn y_i chỉ phụ thuộc vào các nhãn láng giềng trực tiếp y_{i-1} và y_{i+1}, với điều kiện là chuỗi quan sát $\mathbf{H}$[14], như sau:

$$\begin{array}{ccccccccccccc} y_1 & — & \cdots & — & y_{i-1} & — & y_i & — & y_{i+1} & — & \cdots & — & y_m \\ | & & & & | & & | & & | & & & & | \\ \mathbf{h}_1 & & \cdots & & \mathbf{h}_{i-1} & & \mathbf{h}_i & & \mathbf{h}_{i+1} & & \cdots & & \mathbf{h}_m \end{array}.$$

Trong CRF, ta giả định rằng các biến trong đồ thị chỉ phụ thuộc vào các biến láng giềng của chúng. Do đó, $f_j(y_i, y_{i-1}, \mathbf{h}_i)$ có thể được định nghĩa tùy theo cách y_i được kết nối. Nhìn chung có hai loại đặc trưng.

- **Đặc trưng kiểu chuyển** (*Transition-like Features*). Các đặc trưng này mô hình hóa mối liên hệ giữa các nhãn liên tiếp (y_{i-1}, y_i), cho bởi:

  $$f_1(y_i, y_{i-1}, \mathbf{h}_i) = u(y_{i-1}, y_i), \tag{4.83}$$

  trong đó $u(y_{i-1}, y_i)$ là một phần tử của ma trận trọng số $\mathbf{u}$, được đánh chỉ số bởi (y_{i-1}, y_i).

- **Đặc trưng kiểu phát xạ** (*Emission-like Features*). Loại đặc trưng thứ hai mô hình hóa mối liên hệ giữa một nhãn y_i và đầu vào tương ứng $\mathbf{x}_i$, cho bởi:

  $$f_2(y_i, y_{i-1}, \mathbf{h}_i) = g_i(y_i), \tag{4.84}$$

[14]CRF nhìn chung có thể được xếp vào loại **mô hình đồ thị vô hướng** (*undirected graphical models*). Chúng định nghĩa một đồ thị trên một tập biến quan sát được và một tập biến không quan sát được. Các biến này được nối với nhau theo một cách nào đó để tạo thành một đồ thị.

trong đó $g_i(y_i)$ là phần tử thứ y_i của một vectơ $\mathbf{g}_i \in \mathbb{R}^{|V_y|}$. Vectơ $\mathbf{g}_i$ biểu diễn các trọng số gắn $\mathbf{h}_i$ với từng nhãn, dưới dạng

$$\mathbf{g}_i = \mathbf{h}_i \cdot \mathbf{v}, \tag{4.85}$$

trong đó $\mathbf{v} \in \mathbb{R}^{d_h \times |V_y|}$ là một ma trận trọng số.

Để đơn giản hóa ký hiệu, ta dùng y_i (hoặc y_{i-1}) để chỉ biểu diễn one-hot của một nhãn[15]. Khi đó, thay các hàm đặc trưng ở trên vào Phương trình (4.82), hàm tính điểm có thể được viết dưới dạng

$$\begin{aligned}
\text{Score}(y_1 \cdots y_m, \mathbf{h}_1 \cdots \mathbf{h}_m) &= \sum_{i=1}^{m} u(y_{i-1}, y_i) + g_i(y_i) \\
&= \sum_{i=1}^{m} y_{i-1} \cdot \mathbf{u} \cdot y_i^\top + \mathbf{h}_i \cdot \mathbf{v} \cdot y_i^\top \\
&= \sum_{i=1}^{m} (y_{i-1} \cdot \mathbf{u} + \mathbf{h}_i \cdot \mathbf{v}) \cdot y_i^\top .
\end{aligned} \tag{4.86}$$

Chú giải của người dịch: Vectơ one-hot "chọn" phần tử

Giả sử có 3 nhãn {danh từ, động từ, tính từ}. Vectơ one-hot của "động từ" là $(0, 1, 0)$: chỉ có một số 1 tại vị trí của nhãn đó. Nhân một vectơ one-hot với ma trận có tác dụng "rút" ra đúng một hàng/cột: nếu y_{i-1} là one-hot của nhãn a và y_i là one-hot của nhãn b thì $y_{i-1} \cdot \mathbf{u} \cdot y_i^\top$ đúng bằng phần tử $u(a, b)$ ở hàng a, cột b của $\mathbf{u}$. Tương tự, $\mathbf{g}_i \cdot y_i^\top$ chính là $g_i(y_i)$. Vì vậy dòng thứ hai của Phương trình (4.86) chỉ là cách viết lại dòng thứ nhất bằng phép nhân ma trận — loại phép toán mà mạng nơ-ron tính rất nhanh. Ma trận $\mathbf{u}$ có thể hiểu là "bảng điểm chuyển": $u(a, b)$ lớn nghĩa là nhãn b hay đứng sau nhãn a.

Vế phải của phương trình chỉ gồm các phép toán đại số đơn giản trên vectơ và ma trận, cho phép xem mô hình này như một mạng nơ-ron tiêu chuẩn. Nhờ vậy, việc cài đặt hệ thống gán nhãn chuỗi bằng các bộ công cụ mạng nơ-ron khác nhau trở nên thuận tiện. Ta chỉ cần xếp một mạng CRF lên trên một mạng mã hóa và học toàn bộ mạng như thường lệ. Ví dụ, có thể huấn luyện hệ thống này bằng phương pháp hợp lý cực đại, và tối ưu hàm mất mát bằng hạ gradient. Lưu ý rằng, giống như các mô hình

[15] Trong trường hợp này, $y_i \in \mathbb{R}^{|V_y|}$, dù ban đầu nó được dùng như một số vô hướng.

dạng chuỗi hay dạng lưới khác, mạng CRF có thể chạy hiệu quả vì có các thuật toán quy hoạch động, gọi là các phương pháp tiến–lùi (*forward-backward*), để tính cả $\text{Score}(y_1 \cdots y_m, \mathbf{h}_1 \cdots \mathbf{h}_m)$ lẫn $Z(\mathbf{h}_1 \cdots \mathbf{h}_m)$. Bạn đọc quan tâm có thể tham khảo các bài báo liên quan để xem thảo luận chi tiết hơn [Lafferty và cộng sự, 2001; Sutton và McCallum, 2012].

Một ưu điểm chính của việc tích hợp các biểu diễn phân tán vào gán nhãn chuỗi là nó loại bỏ nhu cầu thiết kế đặc trưng thủ công. Thay vào đó, mô hình tự học từ đầu vào thô những đặc trưng hiệu quả nhất cho việc tối ưu mục tiêu gán nhãn chuỗi. Kiến trúc như vậy đã được dùng làm mô hình xương sống cho một số hệ thống tiên tiến nhất [Huang và cộng sự, 2015; Lample và cộng sự, 2016; Ma và Hovy, 2016; Li và cộng sự, 2020c].

4.5.4 Mô hình lai cho mô hình hóa ngôn ngữ

Như đã nói, nhiều bài toán mô hình hóa chuỗi có thể được giải quyết bằng mô hình dựa trên RNN hoặc dựa trên CNN. Mỗi kiến trúc có đặc điểm riêng: RNN ban đầu được thiết kế để xử lý dữ liệu thời gian có độ dài thay đổi, trong khi CNN hiệu quả hơn trong việc nắm bắt thông tin cục bộ trong các vùng giới hạn của đầu vào. Ở đây, ta xét một cách tiếp cận lai cho mô hình hóa ngôn ngữ để tận dụng lợi thế của cả hai.

Nhắc lại từ Mục 4.2.1 rằng một mô hình ngôn ngữ nơ-ron được huấn luyện để dự đoán một phân phối xác suất trên bộ từ vựng khi biết một biểu diễn nào đó của các từ đứng trước. Gọi $w_1 \cdots w_m$ là một chuỗi từ mà ta muốn tính xác suất. Trước hết, ta biểu diễn mỗi từ w_i bằng một vectơ từ $\mathbf{x}_i$. Sau đó, một mô hình RNN xử lý lần lượt từng vectơ từ và xuất ra xác suất:

$$\begin{aligned} \Pr(w_{i+1}|w_1 \cdots w_i) &= \Pr(w_{i+1}|\mathbf{x}_1 \cdots \mathbf{x}_i) \\ &= \Pr(w_{i+1}|\mathbf{h}_i), \end{aligned} \tag{4.87}$$

trong đó $\mathbf{h}_i$ là trạng thái của đơn vị hồi quy tại bước i.

Quá trình chuyển các từ từ dạng ký hiệu sang biểu diễn liên tục đóng vai trò quan trọng trong mô hình này. Dù người làm thực tế thường lấy $\mathbf{x}_i$ từ một bảng nhúng từ, cách làm này coi mỗi từ là một khối nguyên vẹn và bỏ qua cấu trúc bên trong của nó. Kết quả là có thể khó học được các vectơ phân biệt cho các từ hiếm trong những ngôn ngữ có bộ từ vựng lớn [Bojanowski và cộng sự, 2017].

Ở đây ta xét một cách khác để biểu diễn từ. Ý tưởng rất đơn giản: dùng thêm một mạng nơ-ron để nhúng từ [Ling và cộng sự, 2015; Kim và cộng sự, 2016]. Giả sử mỗi từ w_i có thể được biểu diễn thành một chuỗi ký tự. Ta biểu diễn các ký tự này thành các vectơ số thực $\mathbf{z}_{i,1} \cdots \mathbf{z}_{i,\text{len}_i}$ thông qua một bảng nhúng ký tự. Theo công trình của Kim và cộng sự [2016], ta có thể dùng một CNN để biểu diễn $\mathbf{z}_{i,1} \cdots \mathbf{z}_{i,\text{len}_i}$ thành một vectơ từ dưới dạng sau

$$\begin{aligned}\mathbf{x}_i &= \text{CNN}(\mathbf{z}_{i,1} \cdots \mathbf{z}_{i,\text{len}_i}, \mathbf{W}) \\ &= \text{Pooling}(\text{TanH}(\text{Conv}(\mathbf{z}_{i,1} \cdots \mathbf{z}_{i,\text{len}_i}, \mathbf{W}))),\end{aligned} \tag{4.88}$$

trong đó $\text{Conv}(\cdot, \mathbf{W})$ là một tầng tích chập với tham số $\mathbf{W}$, $\text{TanH}(\cdot)$ là hàm tang hyperbol, và $\text{Pooling}(\cdot)$ là một tầng gộp.

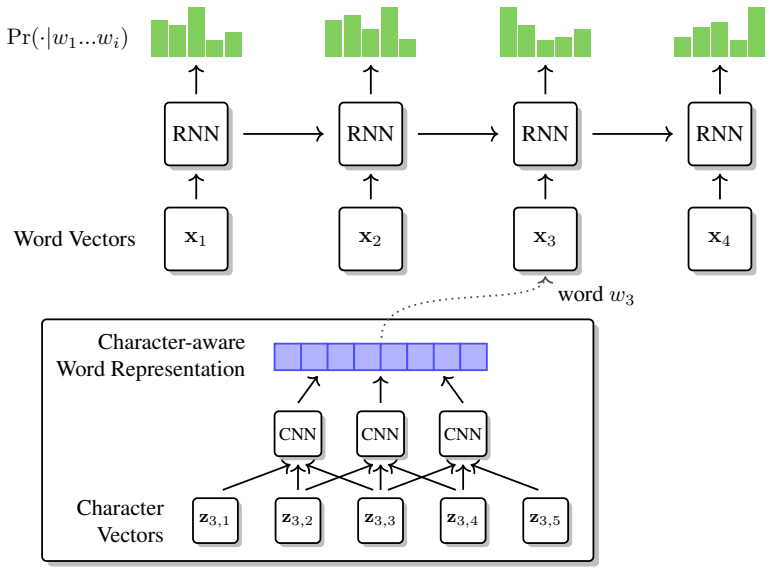
Hình 4.13 trình bày kiến trúc của mô hình. Mô hình này có cấu trúc phân cấp, trong đó các ký tự tạo thành từ, và các từ tạo thành câu hay cụm từ. Xét từ góc độ thực tiễn, trong nhiều bài toán NLP, việc xét đến bản chất phân cấp của ngôn ngữ là khá tự nhiên. Chúng ta sẽ thấy một vài ví dụ về việc tận dụng mối quan hệ giữa các cấp độ biểu diễn ngôn ngữ khác nhau ở các chương sau.

Chú giải của người dịch: Lợi ích của biểu diễn mức ký tự

Giả sử mô hình đã gặp "teach" (dạy) và "teacher" (giáo viên) nhiều lần nhưng chỉ gặp "teachers" một lần. Với bảng nhúng từ thông thường, "teachers" là một mục riêng, học được rất ít. Với CNN trên ký tự, các bộ lọc nhận ra những cụm ký tự quen thuộc như "teach", "-er", "-s", nên vectơ của "teachers" tự nhiên gần với "teacher". Tiếng Việt cũng có hiện tượng tương tự ở mức âm tiết: "giáo viên", "giáo sư", "giáo trình" chia sẻ âm tiết "giáo".

4.6 Tóm tắt

Chương này đã giới thiệu các cách tiếp cận mạng nơ-ron hồi quy và mạng nơ-ron tích chập để mô hình hóa chuỗi từ. Một mặt, mạng nơ-ron hồi quy được thiết kế để xử lý dữ liệu dạng chuỗi và có khả năng ứng dụng rộng rãi trong NLP. Để tăng sức mạnh mô hình hóa của các mô hình này, người ta thường dùng các cơ chế nhớ. Đặc biệt, chúng ta đã giới thiệu LSTM và GRU, hai mô hình phổ biến cho các chuỗi dài. Mặt khác, dù mạng nơ-ron



Hình 4.13: Một mô hình ngôn ngữ với biểu diễn từ có xét đến ký tự (*Character-aware Word Representation*) [Kim và cộng sự, 2016]. Là một mô hình ngôn ngữ, mục tiêu của mô hình này là tính xác suất $\Pr(w_{i+1}|w_1 \cdots w_i)$ với mỗi i. Ta biểu diễn mỗi từ w_i bằng một vectơ số thực $\mathbf{x}_i$ (*Word Vectors*). Vectơ này là đầu ra của một CNN nhận vào chuỗi ký tự (*Character Vectors*) tương ứng với từ đó. Sau đó, chuỗi vectơ từ $\mathbf{x}_1 \cdots \mathbf{x}_m$ được dùng làm đầu vào cho một mô hình RNN + Softmax. Tại mỗi vị trí i, mô hình xuất ra một phân phối trên các từ, trong đó phần tử ứng với w_{i+1} cho ta $\Pr(w_{i+1}|w_1 \cdots w_i)$. Cấu trúc phân cấp này tạo ra một cách tiếp cận đa tỉ lệ cho mô hình hóa ngôn ngữ: một câu được mô hình hóa bằng cách xét các từ, và một từ được mô hình hóa bằng cách xét các ký tự.

tích chập thường được dùng để xử lý dữ liệu hình ảnh, chúng có thể được áp dụng trực tiếp cho mô hình hóa chuỗi. Ta đã thấy rằng tất cả các mô hình này đều có thể dùng trong nhiều bài toán xử lý ngôn ngữ và tiếng nói, bao gồm phân loại văn bản, nhận dạng tiếng nói, gán nhãn chuỗi và mô hình hóa ngôn ngữ.

Nguồn gốc của việc mô hình hóa chuỗi các đơn vị ngôn ngữ có thể truy về những công trình ban đầu trong nhiều lĩnh vực khác nhau. Ví dụ, quá trình sinh một chuỗi từ có thể được mô tả như một xích Markov (*Markov chain*), trong đó việc dự đoán một từ chỉ phụ thuộc vào một số lượng hữu hạn các từ đứng trước [Markov, 1913]. Ý tưởng này thúc đẩy các phương pháp n-gram cho mô hình hóa chuỗi [Shannon, 1948a], cũng như các mô hình Markov ẩn, vốn xuất hiện sau đó và trở nên phổ biến trong việc mô hình hóa chuỗi các cặp biến quan sát được và không quan sát được [Baum và Petrie, 1966; Baum và cộng sự, 1970]. Các mô hình này và các biến thể của chúng đã đặt nền móng cho nhiều hệ thống NLP thành công trong những thập kỷ qua [Manning và Schütze, 1999; Jurafsky và Martin, 2008].

Ý tưởng dùng mạng nơ-ron trong mô hình hóa chuỗi cũng đã được nghiên cứu từ khá lâu. Một ví dụ minh họa cách mạng nơ-ron được phát triển và áp dụng cho mô hình hóa chuỗi là nhận dạng tiếng nói [Lippmann, 1989]. Hầu hết các nghiên cứu thời kỳ đầu của lĩnh vực này hoặc cố gắng kết hợp mạng nơ-ron với các mô hình sẵn có [Bourlard và Wellekens, 1990; Bourlard và Morgan, 1993; Trentin và Gori, 2001], hoặc giải quyết các bài toán con của nhận dạng tiếng nói [Tank và Hopfield, 1987; Waibel và cộng sự, 1989; Lang và cộng sự, 1990; Bengio, 1991]. Tuy nhiên, việc mở rộng quy mô mạng nơ-ron rất khó khăn vì huấn luyện các mạng nơ-ron sâu đòi hỏi nhiều tài nguyên tính toán và dữ liệu. Lĩnh vực này từ lâu đã bị chi phối bởi các cách tiếp cận dựa trên mô hình Markov ẩn và mô hình hỗn hợp Gauss (*Gaussian mixture models*) (GMM), với một quy trình gồm nhiều mô-đun cần tinh chỉnh cẩn thận. Mặt khác, dù các cách tiếp cận hoàn toàn bằng mạng nơ-ron chưa phải là tiên tiến nhất vào thời điểm đó, các nhà nghiên cứu đã nhận thấy tiềm năng của chúng trong việc học biểu diễn của đầu vào âm thanh và giải phóng chúng khỏi các đặc trưng thiết kế thủ công [LeCun và Bengio, 1995].

Một sự chuyển dịch mạnh mẽ từ các cách tiếp cận dạng quy trình nhiều khâu truyền thống sang các cách tiếp cận đầu-cuối (*end-to-end*) đã đến cùng với sự hồi sinh của mạng nơ-ron trong những năm 2000 [Hannun

và cộng sự, 2014; Graves và cộng sự, 2013b]. Sự chuyển dịch này có ảnh hưởng lớn đến mức nó đưa nhiều lĩnh vực lại gần nhau hơn bao giờ hết. Chẳng hạn, trong cả thị giác máy tính lẫn xử lý tiếng nói, thập kỷ vừa qua đã chứng kiến những bước tiến vượt bậc về hiệu năng nhờ các mạng nơ-ron rất sâu và học đầu-cuối [Hinton và cộng sự, 2006; Graves và cộng sự, 2013b; He và cộng sự, 2016a; Krizhevsky và cộng sự, 2017]. Trong NLP, sự chuyển đổi mô thức bắt đầu với các công trình về nhúng từ [Mikolov và cộng sự, 2013a; Pennington và cộng sự, 2014], và tiếp diễn khi các mô hình biểu diễn chuỗi mạnh hơn được phát triển [Vaswani và cộng sự, 2017]. Những cách tiếp cận đơn giản cho mô hình hóa chuỗi, dù không được bàn sâu trong chương này, là các mô hình kết hợp (*compositional models*) [Janssen, 2012]. Ví dụ, ta có thể dùng mô hình túi từ (*bag-of-words*) để cộng hoặc lấy trung bình các vectơ từ của một chuỗi. Dù có kiến trúc đơn giản, các cách tiếp cận này đạt kết quả khả quan trong nhiều bài toán, cung cấp các mô hình cơ sở mạnh cho các nghiên cứu tiếp theo về những mô hình tiên tiến hơn [Conneau và cộng sự, 2018]. Bước tiếp theo, việc áp dụng các mô hình nơ-ron hồi quy và tích chập cho mô hình hóa chuỗi là khá tự nhiên. Điều này không có gì ngạc nhiên vì các mô hình này đã được nghiên cứu khá kỹ trong các lĩnh vực khác [Lipton và cộng sự, 2015; Li và cộng sự, 2021d; Khan và cộng sự, 2020]. Đặc biệt, mô hình LSTM rất phù hợp để xử lý các chuỗi dài và do đó được các nhà nghiên cứu NLP rất quan tâm [Sundermeyer và cộng sự, 2012; Huang và cộng sự, 2015; Wu và cộng sự, 2016]. Tuy nhiên, nghiên cứu trong lĩnh vực này vẫn đang tiếp diễn. Mô hình hóa chuỗi là một trong những lĩnh vực nghiên cứu sôi động nhất và chưa thấy điểm dừng. Có nhiều mô hình dựa trên các kiến trúc mới và cho hiệu năng mạnh hơn trong nhiều bài toán. Thảo luận thêm về một số mô hình này có thể xem ở Chương 6, 7 và 8.

Lưu ý rằng thuật ngữ *mô hình hóa chuỗi* hiện được dùng theo nhiều cách khác nhau, chỉ các bài toán khác nhau. Trong nhiều trường hợp, người ta hay dùng các thuật ngữ *mã hóa* và *bộ mã hóa* hơn để nhấn mạnh quá trình ánh xạ một chuỗi ký hiệu thành một biểu diễn liên tục. Như đã bàn ở các mục trước, một lợi ích của việc xem mã hóa như một bài toán riêng là ta có thể học một mô hình biểu diễn tổng quát, không phụ thuộc vào nơi ta áp dụng nó. Điều này mở ra cánh cửa cho rất nhiều bộ mã hóa được tiền huấn luyện để học cách biểu diễn nhiều loại dữ liệu khác nhau, như văn

bản [Peters và cộng sự, 2018; Devlin và cộng sự, 2019], tiếng nói [Oord và cộng sự, 2018; Hsu và cộng sự, 2021; Chen và cộng sự, 2022], hình ảnh [Chen và He, 2021; Bao và cộng sự, 2021; He và cộng sự, 2022], và tổ hợp của các phương thức này [Chuang và cộng sự, 2020; Li và cộng sự, 2021c; Kim và cộng sự, 2021]. Một khái niệm liên quan chặt chẽ với mã hóa văn bản là nhúng văn bản (*text embedding*) hay nhúng câu (*sentence embedding*) [Conneau và cộng sự, 2017a; Cer và cộng sự, 2018]. Có thể xem chúng nhìn chung là tương đương. Nói chung, một mô hình nhúng trong NLP là một quá trình biến đổi văn bản đầu vào thành một vectơ duy nhất có số chiều thấp, thay vì tạo ra các chuỗi vectơ theo ngữ cảnh [Kiros và cộng sự, 2015; Hill và cộng sự, 2016].

Trong nhiều bài toán NLP, các hệ thống không nhất thiết có đầu vào và/hoặc đầu ra dạng chuỗi. Ví dụ, trong phân loại văn bản, một hệ thống có thể nhận đầu vào có cấu trúc cây và sinh ra một nhãn [Tai và cộng sự, 2015; Yang và cộng sự, 2016]. Trong trường hợp này, ta cần một cơ chế nào đó để mã hóa các cấu trúc phân cấp. Một cách tiếp cận khác là chuyển cây thành chuỗi (hay *cây tuyến tính hóa*) để có thể dùng trực tiếp các mô hình chuỗi xử lý dữ liệu không có dạng chuỗi [Vinyals và cộng sự, 2015]. Đây là một ý tưởng hấp dẫn vì nó mở ra khả năng phát triển một bộ mã hóa áp dụng được cho mọi loại dữ liệu, nếu đầu vào của bộ mã hóa có thể được tuyến tính hóa theo cách nào đó. Ví dụ, bằng cách biểu diễn một hình ảnh thành một chuỗi các mảnh ảnh, các mô hình chuỗi có thể được áp dụng trực tiếp cho phân loại ảnh, đạt kết quả tiên tiến nhất trên một số bài toán [Chen và cộng sự, 2020a; Dosovitskiy và cộng sự, 2021].

Bản dịch tiếng Việt có chú giải (không chính thức, phi thương mại) của *Natural Language Processing: Neural Networks and Large Language Models*, © 2021–2026 Tong Xiao và Jingbo Zhu, `https://github.com/NiuTrans/NLPBook`. Nguyên tác và bản dịch theo giấy phép CC BY-NC 4.0.